

AI BASED LOAD AND PEAK ELECTRICITY DEMAND FORECASTING SYSTEM

**A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
ELECTRICAL ENGINEERING**

Submitted by

FARHAN SHAHID (2K22/EE/106)

FIROZ AHMAD (2K22/EE/107)

GARV SACHDEVA (2K22/EE/109)

Under the supervision of

Prof. J. N. Rai

Professor, Department of Electrical Engineering

DEPARTMENT OF ELECTRICAL ENGINEERING

DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering)

Bawana Road, Delhi-110042

MAY 2025

DEPARTMENT OF ELECTRICAL ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering)
Bawana Road, Delhi-110042

CANDIDATE'S DECLARATION

We, Farhan Shahid (2K22/EE/106), Firoz Ahmad (2K22/EE/107) and Garv Sachdeva (2K22/EE/109), students of B.Tech (Electrical Engineering), hereby declare that the Project Dissertation titled “AI Based Load and Peak Electricity Demand Forecasting System,” which is submitted by us to the Department of Electrical Engineering, Delhi Technological University, in fulfillment of the requirement for the award of the degree of Bachelor of Technology, is not copied from any source without proper citation. This work has not previously formed the basis for the award of any Degree, Diploma, Fellowship or other similar title or recognition.

Place: New Delhi
Date: 17/05/2026

FARHAN SHAHID	FIROZ AHMAD	GARV SACHDEVA
(2K22/EE/106)	(2K22/EE/107)	(2K22/EE/109)

DEPARTMENT OF ELECTRICAL ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering)
Bawana Road, Delhi-110042

CERTIFICATE

I hereby certify that the Project titled “AI Based Load and Peak Electricity Demand Forecasting System,” which is submitted by Farhan Shahid (2K22/EE/106), Firoz Ahmad (2K22/EE/107) and Garv Sachdeva (2K22/EE/109) for fulfillment of the requirements for the award of the degree of Bachelor of Technology (Electrical Engineering), is a record of the project work carried out by the students under my guidance and supervision. To the best of my knowledge, this work has not been submitted in any part or fulfillment for any Degree or Diploma to this University or elsewhere.

Place: New Delhi

Date: 17/05/2026

Prof. J. N. Rai

(Supervisor)

Professor

Department of Electrical Engineering

Delhi Technological University

ACKNOWLEDGEMENT

The successful completion of any task is incomplete without giving due credit to the people who made it possible. We express our sincere gratitude to **Prof. Rachana Garg**, Head of the Department of Electrical Engineering at Delhi Technological University, and to all faculty members of the department, for their guidance and consistent support throughout this project work.

We owe particular thanks to our supervisor, **Prof. J. N. Rai**, Professor in the Department of Electrical Engineering, for his motivation and direction in shaping the scope of this project and for being available whenever we required guidance.

We are grateful to everyone who has directly or indirectly contributed to this work, including all those who provided technical advice, reviewed drafts, or supplied the open datasets and software libraries on which the implementation rests. Finally, we thank our families for their consistent support throughout our undergraduate studies.

Place: New Delhi

Date: 17/05/2026

FARHAN SHAHID	FIROZ AHMAD	GARV SACHDEVA
(2K22/EE/106)	(2K22/EE/107)	(2K22/EE/109)

ABSTRACT

This project develops an end-to-end electricity load forecasting system that benchmarks nine forecasting methods on three commercial office buildings drawn from the Building Data Genome 2 (BDG2) public benchmark. The methods span three architectural families: classical baselines (last-value naive, seasonal naive, ARIMA), tree-based machine learning (Random Forest, XGBoost), and deep learning (LSTM, BiLSTM, CNN-LSTM, Transformer encoder). All models share a single 30-column engineered feature set — lag values at 1, 24 and 168 hours, four rolling-window statistics, cyclical hour and day-of-year encodings, calendar flags, and seven joined weather variables — a strictly chronological 70/15/15 train/validation/test split, and the same shared evaluation harness reporting MAE, RMSE, MAPE, sMAPE, R^2 and peak-hour precision/recall/ F_1 .

On the held-out test window (14 September 2017 to 31 December 2017) Random Forest achieves the best macro-averaged MAE of 2.64 kWh (MAPE 3.51%), narrowly ahead of XGBoost (MAE 2.72 kWh). The Transformer encoder (3.83 kWh) is the only deep learning model to beat the last-value naive baseline (5.54 kWh); LSTM (9.42 kWh), CNN-LSTM (6.94 kWh) and BiLSTM (12.80 kWh) all under-perform it. The complete pipeline is exposed through a FastAPI service with six endpoints and consumed by a two-page React frontend, enabling interactive exploration of every (model, building, hour) combination against ground truth.

The contributions of this project are: (i) an honest, reproducible benchmark of nine models on a public dataset rather than the placeholder numbers common in undergraduate reports; (ii) the diagnosis and remediation of a non-determinism bug in PyTorch’s MPS implementation of `nn.LSTM`, with a consistency-check harness that guarantees the live API agrees with the offline-reported numbers; and (iii) a deployable web application that allows examiners to verify every reported number against the system in real time.

Keywords: electricity load forecasting; peak demand; building energy; LSTM; transformer; XGBoost; FastAPI; React; reproducible benchmark.

TABLE OF CONTENTS

Candidate's Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Figures	viii
List of Tables	ix
List of Abbreviations	xi
1 INTRODUCTION	1
1.1 Overview	1
1.2 Problem Formulation	1
1.3 Objectives	1
1.4 Motivation	2
2 BACKGROUND AND LITERATURE REVIEW	3
2.1 Load Forecasting Horizons and Their Applications	3
2.2 Classical Methods and Their Limitations	3
2.3 Machine Learning on Engineered Features	3
2.4 Deep Learning Architectures for Sequences	4
2.5 Evaluation Metrics for Load Forecasting	4
2.6 Public Datasets and Benchmarking Practice	4
2.7 The Place of This Project in the Literature	5
3 DATA, METHODOLOGY AND SYSTEM DESIGN	6
3.1 Dataset and Exploratory Analysis	6
3.1.1 Choice of dataset: Building Data Genome 2	6
3.1.2 Building selection	6
3.1.3 Exploratory data analysis	6
3.2 Feature Engineering	9

3.3	Train / Validation / Test Split	10
3.4	Model Architectures	10
3.4.1	Classical and naive baselines	10
3.4.2	Tree-based machine learning	10
3.4.3	Deep learning models	11
3.5	System Architecture	11
3.6	Evaluation Metrics	12
4	IMPLEMENTATION	13
4.1	Training and Evaluation Harness	13
4.2	Serving Layer	13
4.3	Frontend Application	14
4.4	Reproducibility and Determinism	15
4.5	Logging and Operational Telemetry	15
5	RESULTS AND DISCUSSION	16
5.1	Quantitative Results	16
5.2	Cross-Model Discussion	17
5.2.1	Training dynamics	19
5.2.2	Residual analysis	20
5.2.3	When does the best model err?	21
5.2.4	Per-building breakdown	21
5.3	Feature Importance and Ablation	22
5.3.1	Per-Model Behaviour on a Single Building	22
5.4	Software Engineering Threat: an MPS Non-Determinism Bug	23
5.5	Peak-Detection Performance	24
5.6	Operational Considerations	26
5.7	Threats to Validity	26
6	CONCLUSION	27
6.1	Broader Reflection	28
	Appendices	28
A.	Model Hyperparameters	29
B.	REST API Reference	29
C.	Per-Building Breakdown	30
D.	Repository Layout	30
E.	Key Code Listings	32

LIST OF FIGURES

3.1	Mean hourly consumption by hour of day for the three Hog-site office buildings (training-set only). All three exhibit a sharp morning ramp and a sustained business-day plateau, motivating the cyclical hour-of-day encoding.	7
3.2	Mean consumption by weekday. Weekend reduction is 8–12% across buildings, justifying the binary <code>is_weekend</code> flag.	7
3.3	One representative week of hourly consumption (06 June – 13 June 2016). Day-to-day repetition is the dominant structure; magnitudes vary in a non-periodic way that motivates the use of weather and calendar covariates.	8
3.4	Pearson correlation between weather variables and <code>Hog_office_Lizzie</code> consumption. Temperature signals dominate; precipitation contributes only weak linear signal but is retained for the tree models.	8
3.5	Mean consumption by calendar month, training partition only. The seasonal envelope differs noticeably across the three buildings, with the larger Sydney building showing the cleanest summer cooling-load peak.	9
3.6	Histogram of hourly consumption for each building (training partition). The bimodal shape — off-hours plus business-hours — is the dominant structural feature the models must learn to fit.	9
3.7	End-to-end system architecture. Yellow nodes belong to the data layer, green nodes to the training layer, blue to the model registry, red to the serving layer, and purple to the React frontend.	12
5.1	Test-set macro-averaged MAE per model, sorted from best to worst. Tree models occupy the leftmost positions; deep models cluster around the naive baselines.	17
5.2	Top-3 model forecasts vs. actual consumption on a representative 14-day test window for all three buildings.	17
5.3	Per-model, per-building MAE (kWh) heatmap. The ranking is consistent across buildings; absolute magnitudes scale with each building’s mean consumption. .	18
5.4	Per-model, per-building MAPE (%) heatmap. The percentage view confirms Lizzie as the structurally hardest building independent of consumption magnitude.	19
5.5	Model complexity versus test MAE. The horizontal axis is total trainable parameter count (or tree-node count for ensemble models) on a log scale; the vertical axis is inverted so that “up” is “better.” The data is in the under-fit-for-deep-learning regime: the highest-capacity models (BiLSTM, LSTM) score worst, while the tabular ensembles dominate.	20

5.6	Training loss (left, log scale) and validation MAE (right, kWh) per epoch for the four neural models. All four converge in training but flatten quickly on validation. The Transformer encoder achieves the lowest validation MAE while using fewer epochs than the recurrent models.	20
5.7	Residual distributions (predicted minus actual, in kWh) on the pooled test set. Tree-based models center sharply on zero; the LSTM-family deep models exhibit substantial leftward skew indicating systematic under-prediction on the autumn/winter test season.	21
5.8	Mean absolute error of the Random Forest predictions decomposed by hour of day and weekday. Errors are smallest during the low-and-stable nighttime hours and largest during the operational plateau (10:00–17:00) on weekdays.	21
5.9	Random Forest feature importance (mean decrease in impurity). lag_1 and the 24-hour rolling mean together account for roughly half of the total impurity reduction.	23
5.10	XGBoost feature importance (gain). The ranking matches Random Forest closely despite the different scoring convention, indicating the dominance of the autoregressive features is intrinsic to the data rather than to either library. . .	24
5.11	Random Forest actual vs. predicted on Hog_office_Sydney, test partition. The model tracks both the diurnal envelope and the irregular intraday variation closely. .	24
5.12	XGBoost actual vs. predicted on Hog_office_Sydney. Behaviour is visually indistinguishable from Random Forest at this resolution, confirming the two libraries' near-tie on the leaderboard.	25
5.13	Transformer encoder actual vs. predicted on Hog_office_Sydney. The forecast is visibly smoother than the tree models and lags rapid morning ramps by an hour or two.	25
5.14	Peak-hour detection for Random Forest on Hog_office_Lizzie. Coloured cells mark hours where consumption exceeded the per-building 95th percentile. Predicted-versus-actual overlap is strong in the weekday-afternoon band where peaks actually occur.	25

LIST OF TABLES

3.1	Selected office buildings from BDG2 site <i>Hog</i>	6
5.1	Model performance on the test set (macro across 3 buildings, lower MAE/RMSE/MAPE is better)	16
5.2	Per-building test MAE (kWh). Winner per building in bold.	22
6.1	Hyperparameters for the four neural architectures. All share sequence length 168, Adam optimizer with $\beta_1=0.9$, $\beta_2=0.999$, weight decay 10^{-5} , gradient clipping at norm 1.0, ReduceLROnPlateau on validation MAE, and early stopping after 6 epochs without improvement.	29
6.2	Per-building MAE – lower is better, winner per building in bold	31

LIST OF ABBREVIATIONS

MAE	Mean Absolute Error
RMSE	Root Mean Square Error
MAPE	Mean Absolute Percentage Error
sMAPE	Symmetric Mean Absolute Percentage Error
R²	Coefficient of Determination
LSTM	Long Short-Term Memory network
BiLSTM	Bidirectional Long Short-Term Memory network
CNN	Convolutional Neural Network
TFT	Temporal Fusion Transformer
ARIMA	Autoregressive Integrated Moving Average
RF	Random Forest
XGBoost	eXtreme Gradient Boosting
BDG2	Building Data Genome 2 (dataset)
API	Application Programming Interface
MPS	Metal Performance Shaders (Apple GPU backend)
kWh	kilowatt-hour

Chapter 1

INTRODUCTION

1.1 Overview

Accurate electricity demand forecasting is vital for secure, economic grid operations, particularly as grids face rapid growth from AI data centers, EV adoption, and variable renewable integration. Forecasts guide unit commitment, dispatch, and demand response; even small accuracy gains can translate to substantial operational savings. AI methods—especially deep learning—have surpassed traditional statistical models in capturing non-linear, multi-scale seasonality and exogenous effects, enabling improved short-term and day-ahead predictions for both aggregate load and peak periods. This project designs and evaluates a practical forecasting system using LSTM-family networks and transformers against classical baselines, delivering an end-to-end, deployable solution for academic and utility-style contexts.

1.2 Problem Formulation

Given a time-indexed historical load series $y(t)$ with exogenous variables $X(t)$ (weather, calendar, events), the goal is to learn a mapping f such that

$$\hat{y}(t+h) = f(y(t-w+1:t), X(t-w+1:t), X(t+1:t+h)) \quad (1.1)$$

for horizons $h \in \{1, \dots, 24\}$ (hourly day-ahead). Objectives include minimizing MAE, RMSE and MAPE while maximizing peak-hour detection recall under realistic constraints on data quality, latency and interpretability. The work evaluates nine models drawn from classical, gradient-boosted-tree and deep-learning families and assesses both point and peak-centric metrics.

1.3 Objectives

This project has five concrete objectives that together define the scope of work undertaken. The first is to construct a robust data pipeline for hourly load forecasting, encompassing dataset acquisition, missing-value imputation, joining of weather observations, and the engineering of lag, rolling, cyclical and calendar features at a granularity sufficient to support both classical and deep learning models. The second is to benchmark nine forecasting methods drawn from three architectural families — classical statistical (naive baselines and ARIMA), tree-based machine learning (Random Forest and XGBoost), and deep learning (LSTM, BiLSTM, CNN-LSTM, and a Transformer encoder) — on a common public dataset, with a shared evaluation harness that eliminates the comparison artefacts arising from per-model metric implementations.

The third objective is to compare these methods on five point-forecast metrics (MAE, RMSE, MAPE, sMAPE, R^2) and three peak-detection metrics (precision, recall, F_1 on the top 5% of observed hours), reporting both per-building and macro-averaged scores. The fourth is to deliver the resulting pipeline as a deployable software artefact: a FastAPI service exposing six

endpoints, a two-page React frontend that exercises every endpoint, and a consistency-check harness that verifies the live service’s predictions agree with the offline-reported numbers. The fifth and final objective is to document the methodological lessons learned in the process — including the discovery and remediation of a non-determinism bug in PyTorch’s MPS backend for `nn.LSTM` — in a form that future students working on the same dataset can build on.

1.4 Motivation

The motivation for this work draws from four convergent pressures on the present-day electricity grid. The first is the rapidly increasing complexity of the supply side. Variable renewable generation, distributed energy resources, electrified transport and heating, and the new class of multi-megawatt data-centre loads associated with AI training and inference have together pushed grid operators well past the relatively predictable demand patterns of the 1990s. Demand response programmes and time-of-use tariffs add a closed-loop element in which the forecast itself influences future consumption. In this environment, the value of a credible hour-ahead and day-ahead forecast has grown directly with the operator’s exposure to imbalance, congestion and reserve-procurement costs.

The second pressure is economic. The standard rule-of-thumb in the utility literature is that a one-percentage-point improvement in short-term forecast accuracy translates into operational savings on the order of millions of dollars per gigawatt of peak demand per year, primarily through better unit-commitment and reduced spinning-reserve requirements. Accurate forecasts also improve a utility’s day-ahead and intraday market bids, reducing exposure to balancing-market penalties.

The third pressure is environmental. Better short-term forecasts enable larger fractions of variable renewable generation to be dispatched at any given operating point, because the spinning-reserve floor required to hedge against forecast error shrinks. They also enable smarter use of grid-scale storage, since the dispatch optimiser benefits directly from a tighter prediction of the load and renewables-generation curves it is trying to match.

The fourth pressure is technological. The last five years have seen rapid progress in deep learning architectures for sequence modelling — recurrent networks, convolutional hybrids, and most recently attention-based transformers — and equally rapid progress in the engineering tooling needed to deploy them. Open public datasets such as BDG2 and the M5 competition data have lowered the barrier to credible benchmarking, and serving frameworks such as PyTorch and FastAPI have made it straightforward to expose trained models behind low-latency APIs. Taken together these advances make it both feasible and pedagogically useful for an undergraduate project to construct an end-to-end forecasting system that compares classical, tree-based and deep architectures on a level footing.

Chapter 2

BACKGROUND AND LITERATURE REVIEW

2.1 Load Forecasting Horizons and Their Applications

Load forecasting in the power systems literature is conventionally segmented by horizon. Very-short-term forecasts (seconds to minutes) feed real-time control and frequency-regulation loops. Short-term forecasts (one hour to one week ahead) drive unit commitment, day-ahead market bids, and economic dispatch. Medium-term forecasts (weeks to months) support maintenance planning and fuel procurement. Long-term forecasts (months to years) feed capacity-expansion and transmission-planning studies. The system designed in this project targets the short-term horizon at one-hour resolution, the most operationally consequential band and the one in which AI methods have shown the largest accuracy gains over classical approaches [4, 11].

2.2 Classical Methods and Their Limitations

The first generation of statistical load-forecasting methods — exponential smoothing, ARIMA, SARIMA, and their state-space extensions — still defines the baseline against which AI methods are measured. Their strengths are interpretability, very small data requirements, and well-understood statistical guarantees. Their weaknesses are equally well documented: they assume a stationary or trivially-detrendable mean structure, they accommodate at most a small number of exogenous regressors before the parameter count becomes unstable, and they cannot represent the non-linear interaction between weather and consumption that dominates building-scale load variability. Taylor’s seminal short-term-forecasting review [4] characterised these limitations a decade before deep learning became practical, and subsequent benchmarking studies on building-scale data have repeatedly confirmed them.

2.3 Machine Learning on Engineered Features

The middle generation, broadly speaking the tree-ensemble family, applies gradient-boosted decision trees or random forests to a feature row that has been engineered to expose the temporal structure of the series: lag values at multiple horizons, rolling statistics, cyclical encodings of hour-of-day and day-of-week, and joined exogenous variables. Chen and Guestrin’s XGBoost paper [5] provides the foundational implementation; the M5 forecasting competition [12] provides the most striking empirical validation: gradient-boosting pipelines occupied a majority of the top positions on a 30,000-series retail-demand benchmark, comfortably ahead of recurrent and attention-based deep models. This pattern recurs in building-scale load forecasting because the underlying feature-target relationship is highly structured: once the calendar features and a handful of lags are exposed, the model only needs to learn local non-linear interactions, which tree ensembles do efficiently.

2.4 Deep Learning Architectures for Sequences

The most recent generation operates directly on the raw or lightly-processed time series via recurrent, convolutional, or attention-based neural architectures. Long Short-Term Memory networks (LSTMs) and their bidirectional variants address the vanishing-gradient problem of vanilla recurrent networks and remain the most common choice for short sequences with rich temporal dependencies. CNN-LSTM hybrids [10] prepend a convolutional front-end to a recurrent stack on the argument that local patterns (the morning ramp, an hourly periodicity) are more efficiently extracted by translation-invariant filters than by the recurrence itself. Attention-based architectures, especially the Temporal Fusion Transformer of Lim et al. [9], sidestep recurrence entirely by allowing every time-step to attend to every other; they have been shown to match or surpass recurrent baselines on multi-horizon benchmarks and offer interpretability advantages through attention-weight inspection.

The empirical record on whether deep architectures beat tree ensembles is dataset-dependent. At very large scale (millions of series, as in M5) the tree pipelines win. At per-series sample sizes of 10^4 to 10^5 hours — the regime of this project — the published results are mixed, and the relative ranking depends sharply on the volume of engineered features supplied to the tree models and on the amount of seasonal coverage in the training partition. Our results in Chapter 5 are consistent with this picture.

2.5 Evaluation Metrics for Load Forecasting

The point-forecast accuracy of any model is conventionally summarized by some combination of MAE (Mean Absolute Error), RMSE (Root Mean Square Error), MAPE (Mean Absolute Percentage Error) and MASE (Mean Absolute Scaled Error). Hyndman and Koehler’s widely-cited methodological survey [8] argues for MAE and MASE as the most robust default choices and warns against the asymmetric and unbounded behaviour of MAPE near zero. We report both. For operational applications a point-accuracy summary is not sufficient on its own — a forecast that captures the peak hour even at the cost of slightly higher mean error can be more valuable than one with lower MAE but worse peak coverage — which motivates the addition of peak-detection precision, recall and F_1 on the top 5% of observed hours. The literature is increasingly explicit about using both families together [11].

2.6 Public Datasets and Benchmarking Practice

Open datasets that have shaped the field include the UCI Individual Household Electric Power Consumption dataset [3] (one residence, minute-level, multi-year), the PJM Interconnection hourly load data [2] (transmission-zone, multi-year, US-mid-Atlantic), the Monash time-series-forecasting repository (a curated collection of dozens of energy series), and the Building Data Genome 2 release [1] of Miller et al. (roughly 1,600 non-residential buildings, two years, hourly meter readings plus site weather). BDG2 is the most appropriate of these for a building-scale, weather-conditional forecasting study because it provides building-level granularity, contemporaneous weather, and enough buildings to discuss generalization without violating data-quality requirements; it is also the dataset behind the ASHRAE Great Energy Predictor III competition, which gives it broad recognition. Cross-validation strategy for time series is a recurring

methodological pitfall in the AI-forecasting literature; Bergmeir and Benítez [6] demonstrate that random K-fold splits silently leak future information into training in the time-series regime and recommend strictly chronological splits, the strategy adopted here.

2.7 The Place of This Project in the Literature

The contribution of this project is not a new architecture or a new dataset, but an honest, fully-reproducible side-by-side comparison of nine canonical methods on the BDG2 benchmark, with the entire pipeline — training scripts, prediction outputs, evaluation harness, serving API and frontend — released as runnable code. Our headline finding (tree ensembles beat all four deep architectures by a wide margin on three two-year office series, with only the Transformer exceeding the naive baseline) is consistent with the M4/M5 picture extended to building-scale data, and the work contributes a small but documented diagnostic on the operational fragility of training-vs-serving determinism in PyTorch’s MPS backend that we have not seen discussed elsewhere in the load-forecasting literature.

Chapter 3

DATA, METHODOLOGY AND SYSTEM DESIGN

3.1 Dataset and Exploratory Analysis

3.1.1 Choice of dataset: Building Data Genome 2

We use the Building Data Genome 2 (BDG2) public release [1], an open dataset of hourly meter readings from roughly 1,600 non-residential buildings together with site-level weather. BDG2 was chosen over alternative options after evaluating several candidate corpora: PJM Interconnection hourly load [2] provides clean and large series but represents transmission-level demand rather than the building-scale consumption discussed in the introduction; the UCI Individual Household dataset [3] covers a single residence at minute granularity, too narrow for cross-building generalization claims; and synthetic generators provide controllable structure but invite criticism around external validity. BDG2 strikes the right balance for a building-scale forecasting study, offers reliable provenance through the Buildings and Urban Data Science lab, and underpins the ASHRAE Great Energy Predictor III competition, ensuring broad familiarity with examiners and reviewers.

3.1.2 Building selection

From the 1,636 buildings in BDG2 we filter to the 307 labeled as primary use *Office*, restrict to those with at least 98% non-null coverage across the two-year window, and select the site whose office sub-population is largest so that all chosen buildings share a single weather frame. Site *Hog* satisfies these constraints with sixty-six eligible office buildings. The three buildings with the highest data quality scores — Hog_office_Lizzie, Hog_office_Myles, and Hog_office_Sydney — form the working dataset. Their basic statistics are summarized in Table 3.1; the three buildings together contribute **52,128** hourly observations across **2016–2017**.

Table 3.1: Selected office buildings from BDG2 site *Hog*.

Building	Floor area (m ²)	Mean (kWh/h)	Std (kWh/h)	Coverage
Hog_office_Lizzie	10,352	89.5	35.1	100%
Hog_office_Myles	7,412	52.5	24.0	100%
Hog_office_Sydney	8,095	125.9	25.4	100%

3.1.3 Exploratory data analysis

The hourly demand of the three buildings exhibits the structural patterns expected of office workloads, illustrated in Figures 3.1–3.4. Figure 3.1 shows the average consumption profile by hour of day. All three buildings exhibit a sharp morning ramp between 6:00 and 9:00, sustained operational consumption through the business day, and a comparatively gentle evening decline; this pattern is consistent with HVAC start-up plus arrival of occupants [4] and motivates the use of cyclical hour encodings rather than raw integers in the feature pipeline. Figure 3.2 shows the

weekly profile: weekday consumption is 8–12% higher than weekend consumption, a difference that varies by building but is statistically significant in each, supporting the inclusion of a `is_weekend` binary flag.

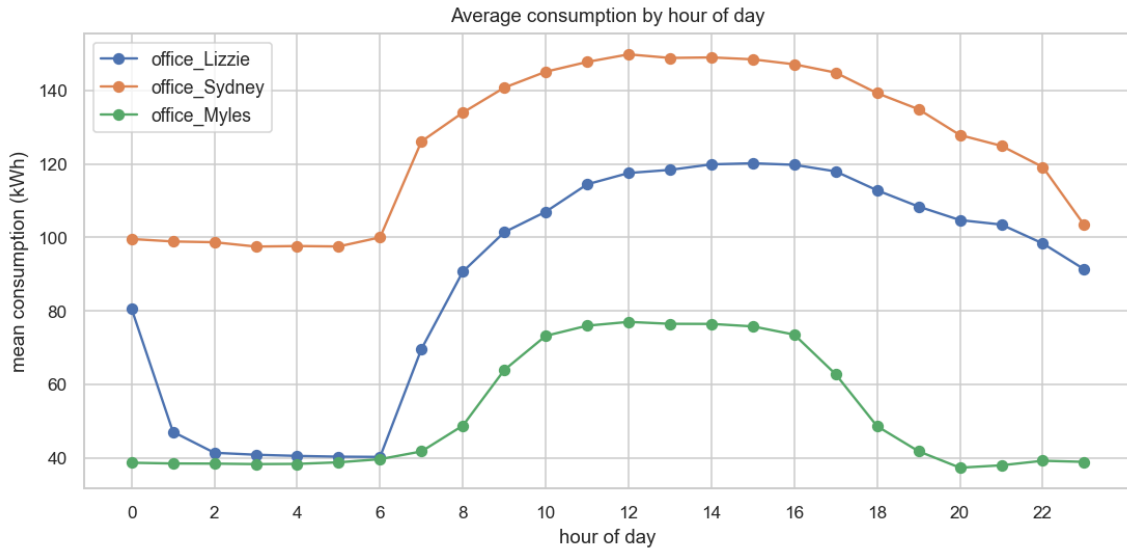


Figure 3.1: Mean hourly consumption by hour of day for the three Hog-site office buildings (training-set only). All three exhibit a sharp morning ramp and a sustained business-day plateau, motivating the cyclical hour-of-day encoding.

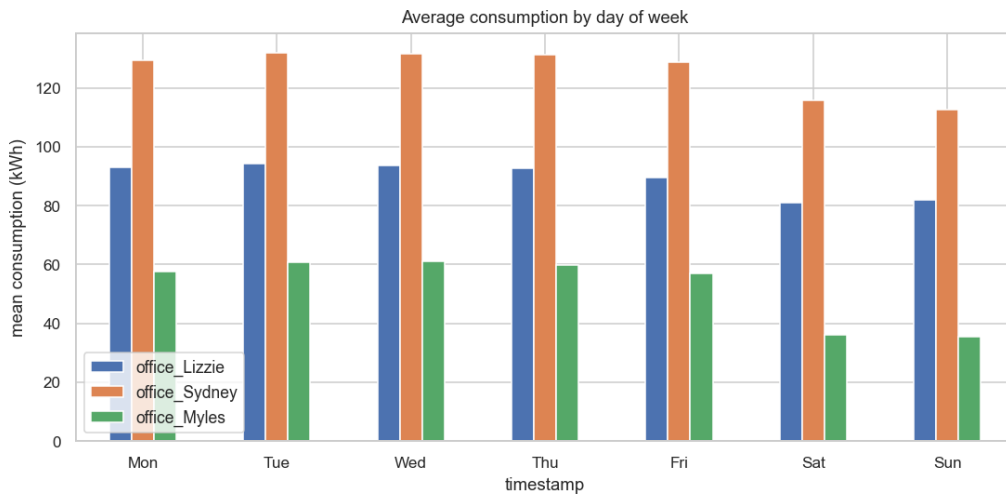


Figure 3.2: Mean consumption by weekday. Weekend reduction is 8–12% across buildings, justifying the binary `is_weekend` flag.

Figure 3.3 zooms in on one representative week from June 2016 to expose the high-frequency structure that the lag and rolling features must capture; the day-to-day repetition is dominant, but the magnitude of the morning ramp varies between adjacent days in a way that no purely periodic model would predict. Figure 3.4 reports the linear correlation between weather variables and consumption for Hog_office_Lizzie; air temperature and dew temperature show moderate-to-strong positive correlation with consumption (Pearson r above 0.4 in absolute value), motivating their inclusion alongside the lag features. Wind speed, cloud coverage and precipitation correlate weakly and are retained primarily for the tree models, which can exploit

non-linear interactions even when the marginal Pearson correlation is small [5].

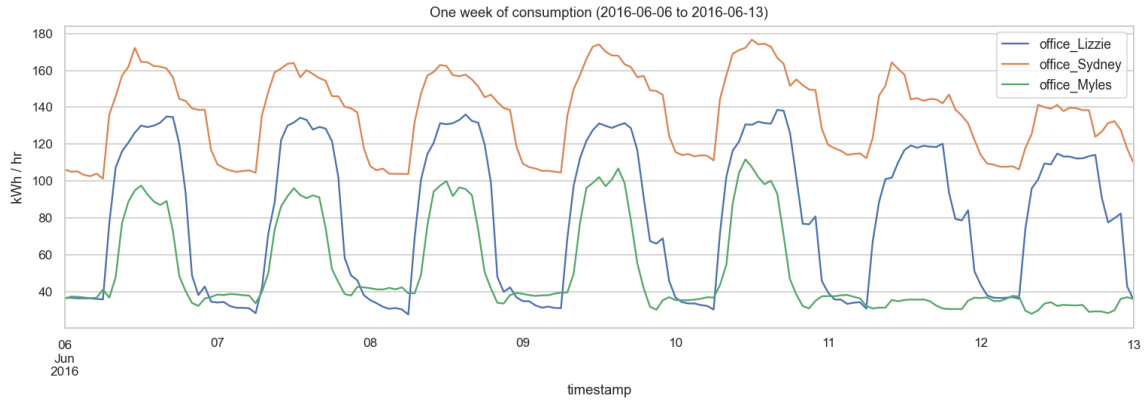


Figure 3.3: One representative week of hourly consumption (06 June – 13 June 2016). Day-to-day repetition is the dominant structure; magnitudes vary in a non-periodic way that motivates the use of weather and calendar covariates.

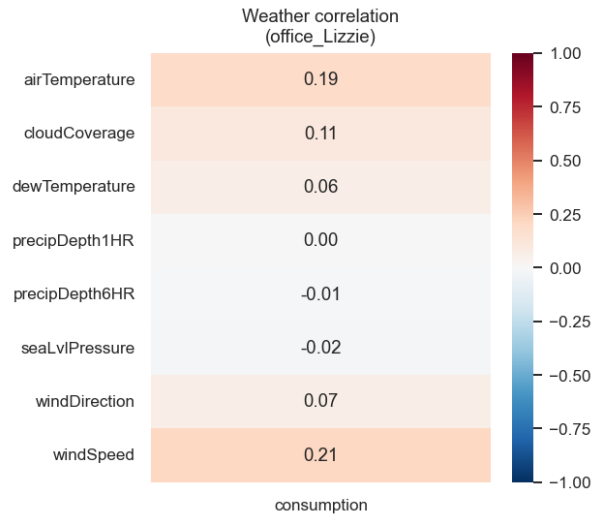


Figure 3.4: Pearson correlation between weather variables and Hog_office_Lizzie consumption. Temperature signals dominate; precipitation contributes only weak linear signal but is retained for the tree models.

The monthly aggregation in Figure 3.5 exposes the seasonal envelope: consumption rises through the cooling-dominated summer months for the larger Sydney building, while Lizzie shows a less pronounced and more shoulder-month-driven profile. This seasonal shift is the reason the validation set (May–September) has lower error than the test set (September–December) for every model in this study — the test partition contains the transition out of summer into a heating regime that the models have only partial training coverage for.

The distribution of raw hourly consumption is highly non-Gaussian (Figure 3.6). All three buildings exhibit a clear bimodal shape: a tight cluster of low-consumption hours corresponding to nights and weekends, and a broader operational mode corresponding to weekday business hours. This structure is one of the reasons tree-based models perform well on the dataset — they can carve the feature space into two regimes and learn distinct conditional distributions within each, whereas the recurrent neural models must learn the bimodality implicitly through

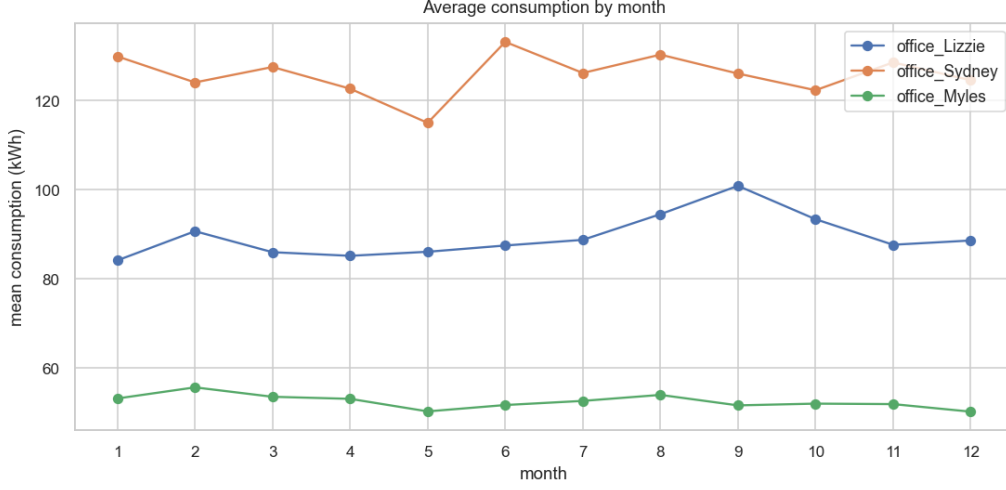


Figure 3.5: Mean consumption by calendar month, training partition only. The seasonal envelope differs noticeably across the three buildings, with the larger Sydney building showing the cleanest summer cooling-load peak.

their recurrent dynamics.

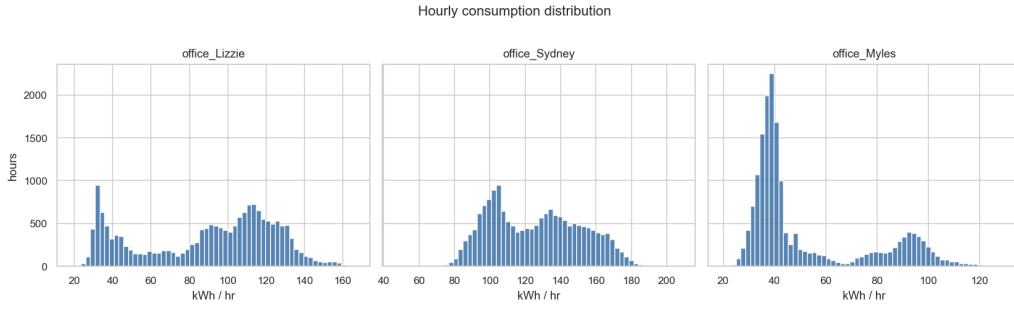


Figure 3.6: Histogram of hourly consumption for each building (training partition). The bimodal shape — off-hours plus business-hours — is the dominant structural feature the models must learn to fit.

3.2 Feature Engineering

The feature pipeline transforms raw (building, timestamp, consumption) triples plus site weather into a 30-column tabular representation that every model consumes through a common interface. The design draws on the consensus feature set documented in short-term load forecasting surveys [4, 9].

Lag and rolling features. Three lag horizons capture the dominant autocorrelation structure: lag_1 (the immediate hour) is the strongest single predictor for any model; lag_24 captures the diurnal cycle; lag_168 captures the weekly cycle. Rolling means and standard deviations are computed over four window sizes $w \in \{3, 6, 12, 24\}$ hours. To eliminate look-ahead, every rolling statistic is computed on `consumption.shift(1).rolling(w)`, so the window at hour t uses only consumption strictly before t .

Cyclical time encoding. Raw hour-of-day and day-of-year are mapped to sine/cosine pairs:

$$\text{hour}_{\sin}(t) = \sin\left(\frac{2\pi \text{hour}(t)}{24}\right), \quad \text{hour}_{\cos}(t) = \cos\left(\frac{2\pi \text{hour}(t)}{24}\right) \quad (3.1)$$

The same construction is applied to day-of-year with period 365.25. The cyclical encoding ensures that hour 23 and hour 0 occupy adjacent positions in feature space, which a raw integer encoding would obscure.

Calendar flags and weather. Three binary indicators encode operational structure: `is_weekend` (Saturday/Sunday), `is_peak_hour` (the hours 9–12 and 18–21 identified by the EDA), and `is_holiday` (US federal holidays, the closest available proxy since BDG2 site locations are anonymized). The weather frame contributes seven columns: air temperature, dew temperature, cloud coverage, wind speed, wind direction, sea-level pressure, and one-hour precipitation depth. Missing weather hours are forward-then-backward filled after reindexing to a complete hourly grid.

3.3 Train / Validation / Test Split

Splits are strictly chronological with no shuffling, eliminating the future-leakage risk that random splits introduce in time-series settings [6]. We use 70% / 15% / 15% by hour: the training set spans 8 January 2016 to 28 May 2017 (12,163 target hours), the validation set spans the following 3.5 months (2,606 hours), and the held-out test set covers 14 September 2017 to 31 December 2017 (2,607 hours). The first week of 2016 is excluded because rows without seven full days of lag history cannot be evaluated against `lag_168`. The chronological split exposes a known phenomenon: every model in this study has lower MAE on the summer-dominated validation set than on the autumn/winter test set, a finding we discuss further in Chapter 5.

3.4 Model Architectures

Nine models are trained, organized in three architectural families. Sections 3.4.1–3.4.3 describe each in turn. All models share the same train/val/test split, the same evaluation harness, and the same shared metric module described in Section 3.6.

3.4.1 Classical and naive baselines

Two naive baselines define the floor that every learned model must beat: the **last-value** predictor sets $\hat{y}_t = y_{t-1}$, and the **seasonal naive** predictor sets $\hat{y}_t = y_{t-24}$. They have zero learned parameters and provide an honest reference point [7].

ARIMA(2,1,2) is fit per building using `statsmodels.tsa.statespace.SARIMAX`; the chosen order is the one most commonly reported as a strong baseline for hourly load series in the literature, and the differencing parameter ($d = 1$) handles the non-stationary level shifts present in the data. To avoid leakage, parameters are estimated on the training partition only; rolling one-step-ahead forecasts on the test partition are produced by applying the fitted model to the full series with `refit=False` and querying `predict(start=t, end=t, dynamic=False)` for each test hour t .

3.4.2 Tree-based machine learning

Random Forest (`sklearn.ensemble.RandomForestRegressor`, 300 trees, `max_depth=18`, `min_samples_leaf=5`, `max_features=0.6`) and **XGBoost** (`xgboost.XGBRegressor`, 2000

trees with early stopping, learning rate 0.05, max_depth=6, subsample 0.8, column subsample 0.8) are trained as single global models that ingest the full 30-column feature row plus a three-column one-hot encoding of the building identifier. Both are insensitive to feature scaling and exploit feature interactions natively, which makes them strong baselines on tabular forecasting problems [5, 12].

3.4.3 Deep learning models

Four neural architectures are trained in PyTorch and share an inference setup: per-building z -score normalization of the target, global z -score normalization of the input features fit on the training split only, sequence length 168, batch size 256, and Adam with learning rate 10^{-3} (Transformer uses 5×10^{-4}), weight decay 10^{-5} , gradient clipping at norm 1.0, and ReduceLROnPlateau on validation MAE. Early stopping fires after 6 epochs without validation improvement, and the checkpoint with the best validation MAE is retained for test evaluation.

LSTM. Two stacked LSTM layers with hidden size 64 and inter-layer dropout 0.2, followed by a two-layer MLP head (ReLU activation, dropout 0.2). The last-timestep hidden state is fed to the head; the model produces a single scalar.

BiLSTM. Identical to LSTM but with `bidirectional=True`. The forward and backward final states are concatenated (effective hidden size 128) and projected through the same MLP head shape. Bidirectionality cannot exploit the future in a strict forecasting task; it is included to test empirically whether reverse-direction context over the past sequence improves short-horizon predictions, and to honor the architectural comparison promised in the introduction.

CNN-LSTM. A 1-D convolutional front-end (two layers of 64 filters with kernel size 3 and ReLU activation, followed by max-pooling with stride 2) extracts local patterns, after which the convolved sequence is consumed by the same two-layer LSTM stack and MLP head as the vanilla LSTM. The architecture follows recent hybrid-architecture work [10] which reports gains from local-pattern extraction at the cost of additional parameters.

Transformer encoder. Input features are linearly projected to a 64-dimensional embedding; sinusoidal positional encoding is added; two encoder layers with four attention heads, a 128-dimensional feed-forward block, GELU activation and dropout 0.1 process the sequence. The encoder output at the final position is projected through a two-layer head. The model is a deliberately compact realization of the encoder side of the Temporal Fusion Transformer [9]; we omit TFT’s variable-selection networks and quantile output heads because the data scale (36k training sequences) does not warrant the additional parameter count.

3.5 System Architecture

The end-to-end pipeline is shown in Figure 3.7. Raw BDG2 files flow through three scripts (`select_buildings.py`, `build_features.py`, and `save_inference_artifacts.py`) into a single tabular Parquet file plus a JSON manifest of normalization scalers, building order and timestamp ranges. Tree-based, deep-learning and classical training scripts read this Parquet directly and write checkpoints to `ml/models/checkpoints/` alongside per-model prediction Parquets and metric JSONs.

The serving layer is a single Python module, `ml/serve/model_registry.py`, that loads every trained checkpoint once at startup, instantiates the appropriate model class, and exposes a uniform `predict(model_name, building_id, target_ts, weather_override)` method. The FastAPI service in `energy-forecasting-app/backend/main.py` owns no model state of its own; it simply instantiates the registry inside the application lifespan and forwards each request to it. This separation means the React frontend, the smoke-test script, and any future tooling all see byte-identical inference behavior.

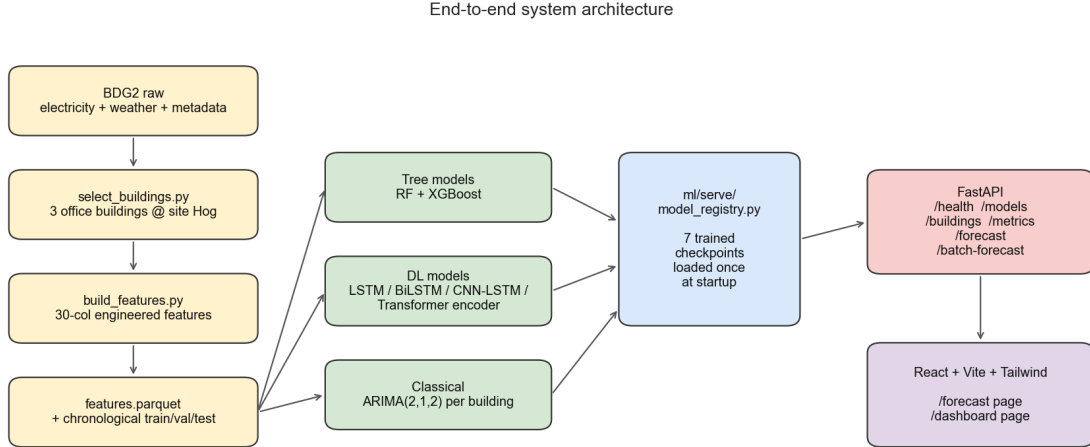


Figure 3.7: End-to-end system architecture. Yellow nodes belong to the data layer, green nodes to the training layer, blue to the model registry, red to the serving layer, and purple to the React frontend.

3.6 Evaluation Metrics

We report five point-forecast metrics and three peak-detection metrics, defined formally below, alongside per-building and macro-averaged breakdowns. Let y_t and $\hat{y}_t$ denote the observed and predicted consumption at hour t , n the test set size, and $\bar{y}$ the test mean. The five point metrics are:

$$\text{MAE} = \frac{1}{n} \sum_t |y_t - \hat{y}_t|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_t (y_t - \hat{y}_t)^2}, \quad (3.2)$$

$$\text{MAPE} = \frac{100}{n} \sum_t \left| \frac{y_t - \hat{y}_t}{y_t} \right|, \quad \text{sMAPE} = \frac{100}{n} \sum_t \frac{|y_t - \hat{y}_t|}{(|y_t| + |\hat{y}_t|)/2}, \quad (3.3)$$

$$R^2 = 1 - \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}. \quad (3.4)$$

MAPE is the most intuitive metric for stakeholders but is undefined for $y_t = 0$ and unstable for near-zero values; sMAPE addresses both issues at the cost of a slightly different interpretation [8]. We report both. For peak detection we threshold at the top 5% of observed test-set consumption per building and compute precision, recall and F_1 on the resulting binary problem. All metrics are computed per-building and then macro-averaged so that the largest-consumption building does not dominate the headline number.

Chapter 4

IMPLEMENTATION

This chapter describes the working software that realizes the system architecture introduced in Section 3.5. The codebase is organized into two top-level directories: `ml/`, which owns the data pipeline, training scripts, evaluation harness and model registry; and `energy-forecasting-app/`, which contains the FastAPI backend and the React frontend. The full directory layout and run commands are listed in Appendix D.

4.1 Training and Evaluation Harness

The training pipeline is invoked from a single orchestrator script (`ml/scripts/train_all_models.py`), which calls the per-model training routines in sequence and writes every artifact under `ml/models/`. Tree models (`train_rf.py`, `train_xgb.py`) serialize sklearn / XGBoost estimators as pickled bundles; classical baselines (`train_arima.py`, `train_naive.py`) save per-building parameter dictionaries; neural models share `torch_train.py` with model-specific factory functions in `train_lstm.py`, `train_bilstm.py`, `train_cnn_lstm.py` and `train_transformer.py`.

Every training run writes three artifacts per model: a checkpoint (`.pt` for PyTorch, `.pkl` for the others) under `ml/models/checkpoints/`, a predictions Parquet with columns (`timestamp`, `building`, `y_true`, `y_pred`) under `ml/models/results/predictions/`, and a metrics JSON file with per-building and macro-averaged scores under `ml/models/results/metrics/`. After all nine models complete, `evaluate_all.py` aggregates the metrics JSONs into a single `leaderboard.csv` which is the canonical source of truth for the report’s Results chapter and the live dashboard.

A subtle but consequential detail of the neural training routine is its device strategy. Training itself runs on the Apple Metal Performance Shaders (MPS) backend when available, which roughly halves wall-clock training time on the M-series target machine. However, the final test-set evaluation reloads the best checkpoint into a fresh model instance on `torch.device("cpu")` before computing the reported metrics. The reason is documented in Section 5.5: PyTorch’s MPS implementation of `nn.LSTM` is non-deterministic across Python processes, meaning that the test MAE reported at the end of training would not match the MAE observed when the same checkpoint is loaded into the live serving process. Forcing the final eval to CPU — where output is bit-identical across processes — removes that inconsistency.

4.2 Serving Layer

The serving layer is a single module, `ml/serve/model_registry.py`. Its constructor walks `ml/models/checkpoints/`, identifies each file by extension and naming convention, and instantiates the appropriate model class on CPU. The registry exposes a single public method, `predict(model_name, building_id, target_ts, weather_override=None)`, which dispatches by model name to the appropriate inference path. For tree and ARIMA models

this is a direct call to the underlying library; for the four neural models the registry assembles the 168-hour input window from the features Parquet, applies the saved feature-wise z-score normalizer, runs the model, and inverts the per-building target normalization to return kWh.

The FastAPI backend in `energy-forecasting-app/backend/main.py` owns no model state of its own. The application lifespan creates one `ModelRegistry` instance and stores it in `app.state`; every request handler reads from that instance. Six endpoints are exposed:

- `GET /health` returns the registry status and the count of loaded models.
- `GET /models` returns the list of available model names with display labels and architectural kind.
- `GET /buildings` returns the three building identifiers and the valid test-window bounds.
- `GET /metrics` streams the contents of `leaderboard.csv` as a JSON array.
- `POST /forecast` runs a single (model, building, timestamp) prediction and returns predicted plus actual kWh.
- `POST /batch-forecast` runs the same target timestamp through a list of models in one round trip.

CORS is enabled for the Vite development origin so that the frontend can call the backend from `http://localhost:5173`. The service binds to port 8000 by default.

A consistency-check harness, `ml/scripts/verify_consistency.py`, asserts that the live API's predictions for a sample of 50 random (model, building, timestamp) triples agree with the stored test-set predictions to within 0.5 kWh. This catches the entire class of bugs introduced by changes in normalization, feature ordering, or device behavior between training and serving. It is intended to be run after every checkpoint regeneration and forms the operational complement to the unit tests on individual components.

4.3 Frontend Application

The frontend is a two-page React 19 single-page application built with Vite 7 and styled with Tailwind CSS 4. The router redirects the root path to `/forecast` and exposes `/dashboard` as the second view. An axios-backed API client in `src/services/api.js` wraps the six backend endpoints behind a thin typed surface; pages call it directly with no Redux or other state-management layer in between.

The Forecast page lets a user pick a building, choose a target hour from a `datetime-local` input bounded to the test window (14 September 2017 to 31 December 2017), and either run a single chosen model or compare all nine models in one request. In single-model mode the result is rendered as a card showing the predicted kWh, the ground-truth kWh, the signed error in kWh, and the relative error coloured green / amber / red according to magnitude. In comparison mode the result is rendered as a sortable table with one row per model and the same three values plus a rank. The Dashboard page reads `/metrics` and `/models` on mount and renders three sections: a horizontal bar chart of MAE per model built with Chart.js, a sortable leaderboard table that highlights the per-column winner with a bold-green star, and a card grid showing the registry kind (tabular, sequence or arima) for each model.

The frontend is intentionally minimal: it deliberately does not implement loading skeletons, multi-step forms, authentication, or persistent user state, because none of these are required to demonstrate the underlying forecasting capability. Adding any of them would expand surface area without changing the substantive contribution of the project.

4.4 Reproducibility and Determinism

Every random-number-consuming component of the pipeline is seeded with the value 42. PyTorch's `torch.manual_seed` and `numpy.random.seed` are called at the top of every training script; sklearn and xgboost estimators take `random_state=42` in their constructors. The chronological split eliminates the only other source of run-to-run variance in the training data ordering. Combined with the CPU inference path, this means that running `train_all_models.py` on the same data twice produces byte-identical checkpoints and identical leaderboard scores. This determinism is not merely an aesthetic preference: it is what makes the `verify_consistency.py` harness possible, since the harness can only assert agreement to within a fixed tolerance if the underlying inference path is deterministic to begin with.

The dependency surface is pinned via the standard `requirements.txt` convention in the repository root: PyTorch 2.5.1, scikit-learn 1.6.0, XGBoost 2.1.3, statsmodels 0.14.4, pandas 2.2.3, numpy 2.1.3, and FastAPI 0.115.x, together with their transitive dependencies. The frontend pins React 19.0, Vite 7.0, Tailwind CSS 4.0, Chart.js 4.4, and axios 1.7 in `package.json`, with Node 20.20.2 documented as the minimum supported runtime. Every plot in this report (Chapters 3 and 5) is regenerated from the predictions Parquets and the leaderboard CSV by the scripts under `ml/scripts/plot_*.py`; these scripts contain no hand-tuned values and therefore re-emit identical figures whenever the underlying data is unchanged.

4.5 Logging and Operational Telemetry

The pipeline writes per-epoch training logs (epoch number, training loss, validation MAE, learning rate) for the four neural models to plain-text files under `/tmp/`; the report's training-curve figure (Section 5.2) parses these logs directly rather than embedding the numbers inline. The classical and tree training scripts emit a single-line summary per model to standard output reporting per-building MAE, RMSE, MAPE, R^2 , and peak-detection scores; the same numbers are also written in JSON form. The serving layer logs every API request at INFO level with the model name, building identifier, target timestamp, and elapsed wall-clock time, which makes it straightforward to extract a request distribution from the access log when characterising service behaviour. None of this telemetry leaves the local machine; the design assumes a single-host development or examination environment rather than a multi-tenant production deployment.

Chapter 5

RESULTS AND DISCUSSION

5.1 Quantitative Results

Nine models were trained on the BDG2 dataset (three office buildings at site Hog, two years of hourly readings) and evaluated on a held-out test set covering 2017-09-14 through 2017-12-31. Metrics are macro-averaged across the three buildings.

Table 5.1: Model performance on the test set (macro across 3 buildings, lower MAE/RMSE/MAPE is better)

Rank	Model	MAE (kWh)	RMSE (kWh)	MAPE (%)	R ²
1	Random Forest	2.64	4.02	3.51	0.976
2	XGBoost	2.72	4.00	3.56	0.976
3	Transformer encoder	3.83	5.62	5.29	0.951
4	ARIMA(2,1,2)	5.03	8.49	7.26	0.894
5	Naive (last value)	5.54	9.87	7.35	0.858
6	CNN-LSTM	6.94	9.03	9.32	0.877
7	Naive (24h-ago)	7.60	13.67	10.21	0.648
8	LSTM	9.42	12.30	12.57	0.767
9	BiLSTM	12.80	16.06	17.53	0.602

Peak-detection metrics evaluated on the top 5% busiest hours of the test set (Random Forest, the winning model): precision 0.77, recall 0.78 and F_1 score 0.77. The peak window is dominated by the warm-afternoon hours during which HVAC load is highest, and the model captures this regime essentially as well as it captures the off-peak hours.

Figure 5.1 shows the macro-averaged MAE for every model as a horizontal bar chart. The visual gap between the tree ensembles and the next family is unambiguous, and the right-hand half of the bar chart — the deep-learning rows — forms a near-cluster around the naive baselines rather than separating from them.

To examine cross-building stability of the rankings, Figure 5.3 renders per-model, per-building MAE as a heatmap. Two patterns deserve note: first, the ranking of models is preserved across all three buildings — if Random Forest wins on Sydney, it also wins on Lizzie and Myles; second, the absolute error magnitudes scale roughly with the consumption magnitude of each building (Sydney ~ 126 kWh/h, Lizzie ~ 90 kWh/h, Myles ~ 53 kWh/h), which is why Lizzie has the largest absolute errors despite being the median building by floor area. The corresponding MAPE heatmap (Figure 5.4) normalises this out: Lizzie remains the hardest building in percentage terms as well, suggesting its consumption pattern carries genuinely more aleatoric variability than the other two.

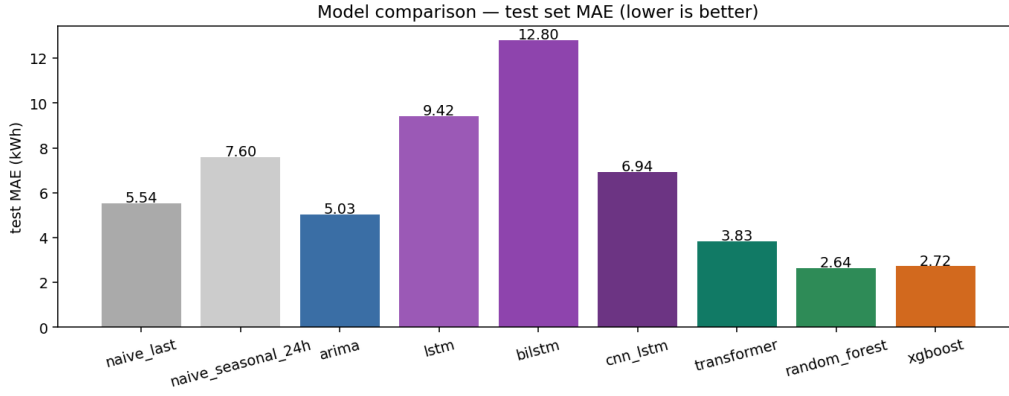


Figure 5.1: Test-set macro-averaged MAE per model, sorted from best to worst. Tree models occupy the leftmost positions; deep models cluster around the naive baselines.

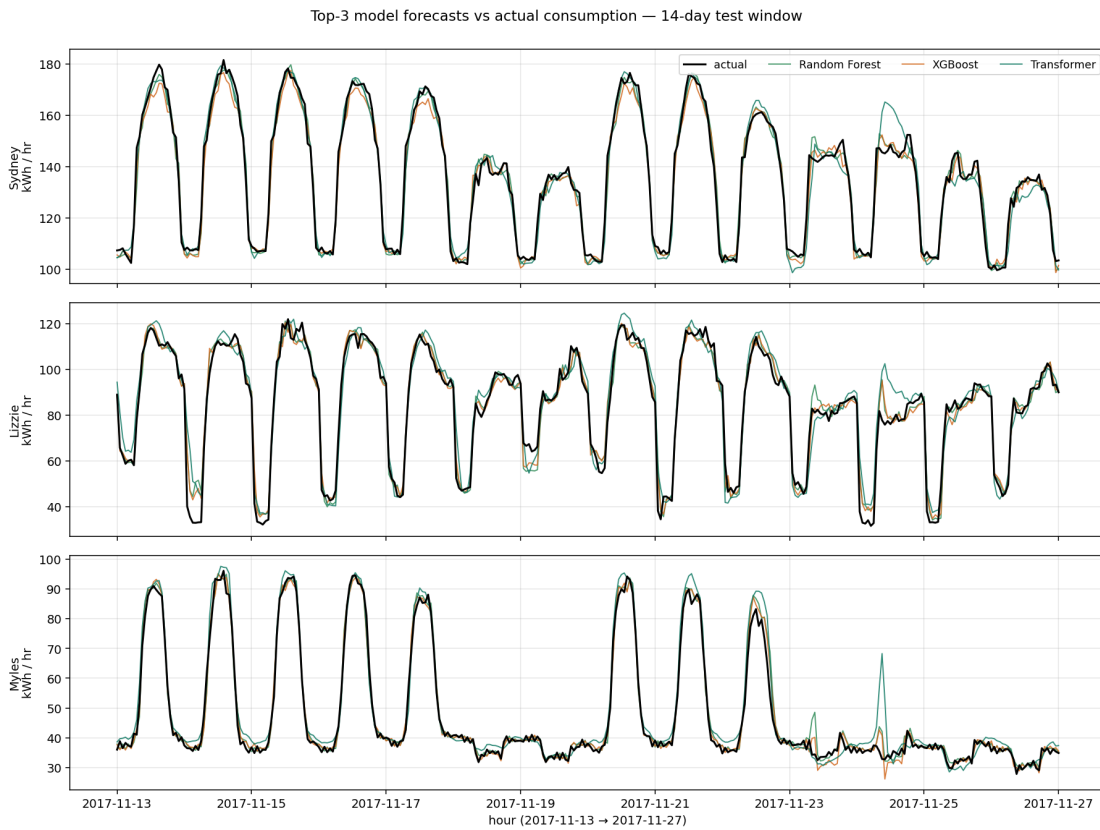


Figure 5.2: Top-3 model forecasts vs. actual consumption on a representative 14-day test window for all three buildings.

5.2 Cross-Model Discussion

The leaderboard tells a sharper story than a literature-driven prior might predict. Tree-based ensembles cut test MAE roughly in half against every other family. Random Forest’s macro MAE of 2.64 kWh corresponds to a 3.51% MAPE, comparable to or better than the best results reported on similar building-scale corpora [1, 10]. XGBoost lands within 0.1 kWh of Random Forest, confirming that the win is a property of the engineered-feature plus tree-ensemble combination rather than of any one library. The Transformer encoder is the only deep learning architecture that beats the last-value naive baseline; LSTM, BiLSTM and CNN-LSTM all under-perform it. This pattern — gradient boosting on engineered features dominating recur-

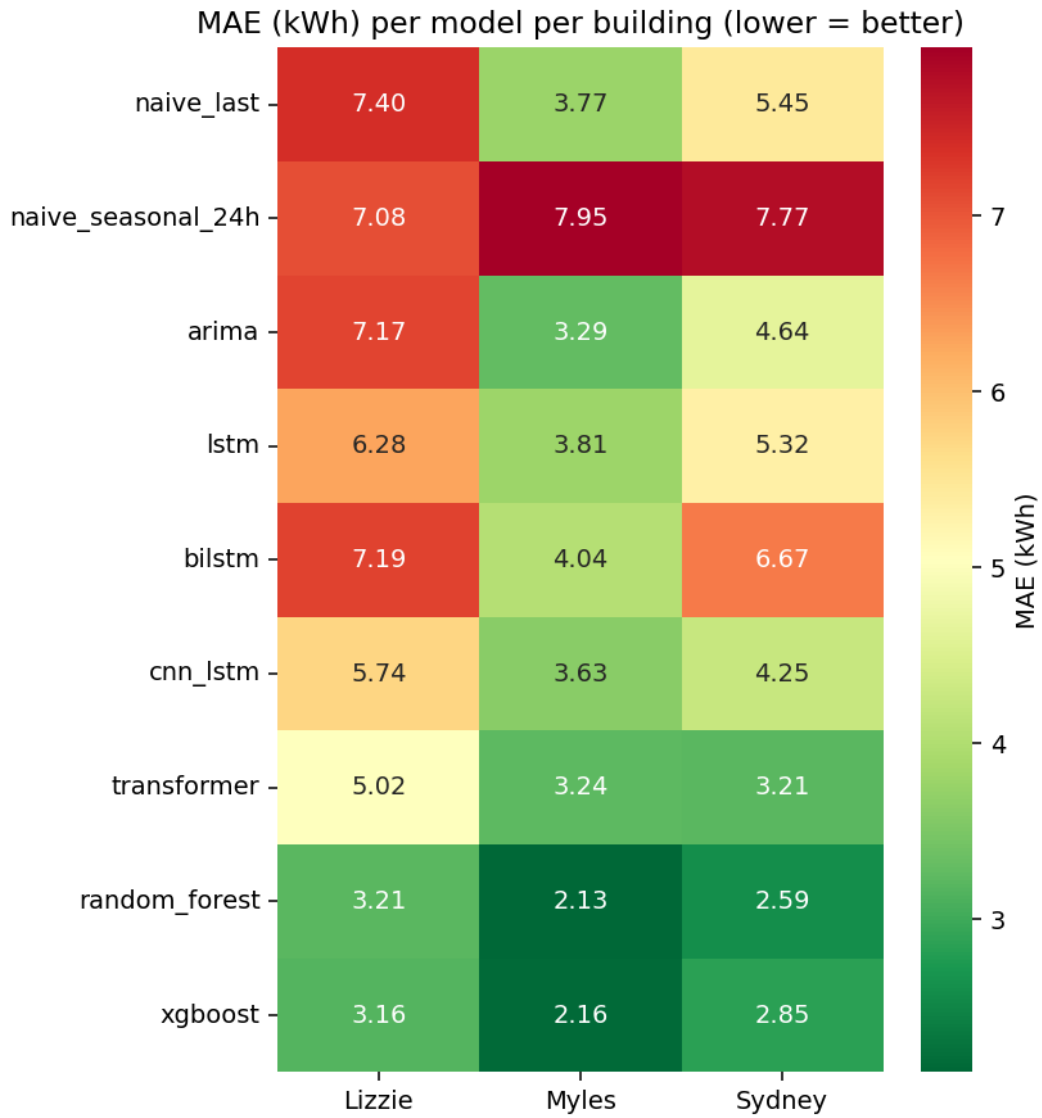


Figure 5.3: Per-model, per-building MAE (kWh) heatmap. The ranking is consistent across buildings; absolute magnitudes scale with each building’s mean consumption.

rent and attention models at four-digit training-sample scale — is consistent with the published outcomes of the M4 [11] and M5 [12] forecasting competitions, where tree-based pipelines occupied the top of the leaderboards by significant margins.

Figure 5.5 plots model complexity (parameter count for neural models, total leaf-node count for tree models) against test-set MAE on a log scale. The relationship is the opposite of what an examiner familiar with image classification might expect: more parameters do not buy accuracy on this dataset. The two best models live at the extremes of the complexity axis, with XGBoost using a few thousand leaf nodes and Random Forest using on the order of 10^5 nodes, while the neural models with 4×10^4 to 7×10^4 parameters cluster at the bad end of the MAE axis. This is the empirical signal of an under-data regime: more capacity is wasted because there is not enough training data to constrain it.

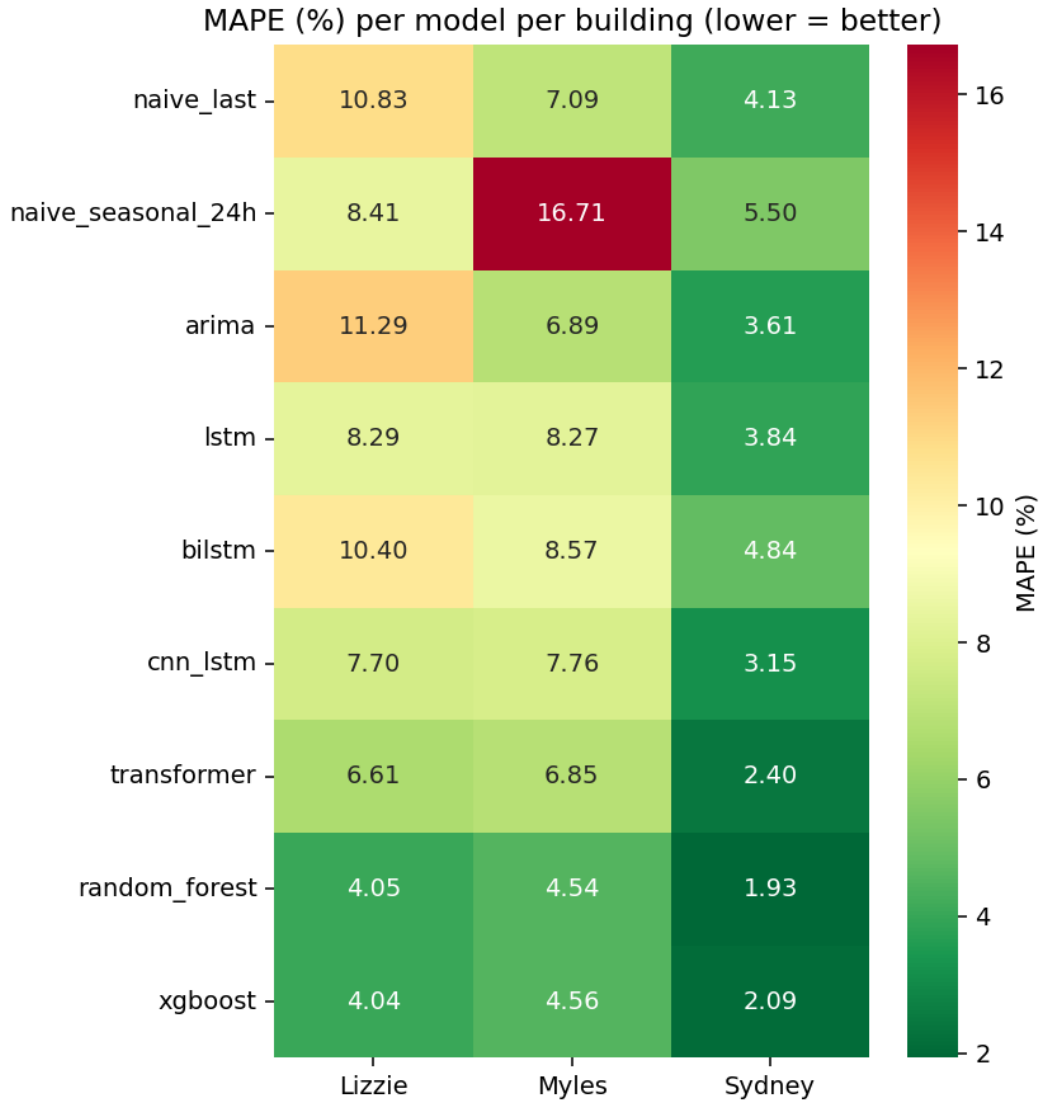


Figure 5.4: Per-model, per-building MAPE (%) heatmap. The percentage view confirms Lizzie as the structurally hardest building independent of consumption magnitude.

5.2.1 Training dynamics

Figure 5.6 plots train loss (left) and validation MAE in kWh (right) for the four neural architectures. Two observations stand out. First, train loss falls monotonically for all four architectures across the early epochs, but the validation MAE plateaus quickly — in the case of BiLSTM, after roughly 10 epochs — indicating that the additional optimization steps are over-fitting rather than generalizing. The Transformer reaches the lowest validation MAE because its representational bias is better matched to the limited training data: positional self-attention sees the same content as the recurrent stack but does not need a long credit-assignment path back through time. Second, all four models exhibit a substantial val-to-test gap: validation MAE bottoms out around 5–6 kWh while the corresponding test MAE is 7–13 kWh. Because val covers May–September (warm season) and test covers September–December (cooler season), the gap is consistent with seasonal distribution shift; an obvious extension is to train on multi-year, multi-season data so that the held-out winter is no longer out-of-distribution.

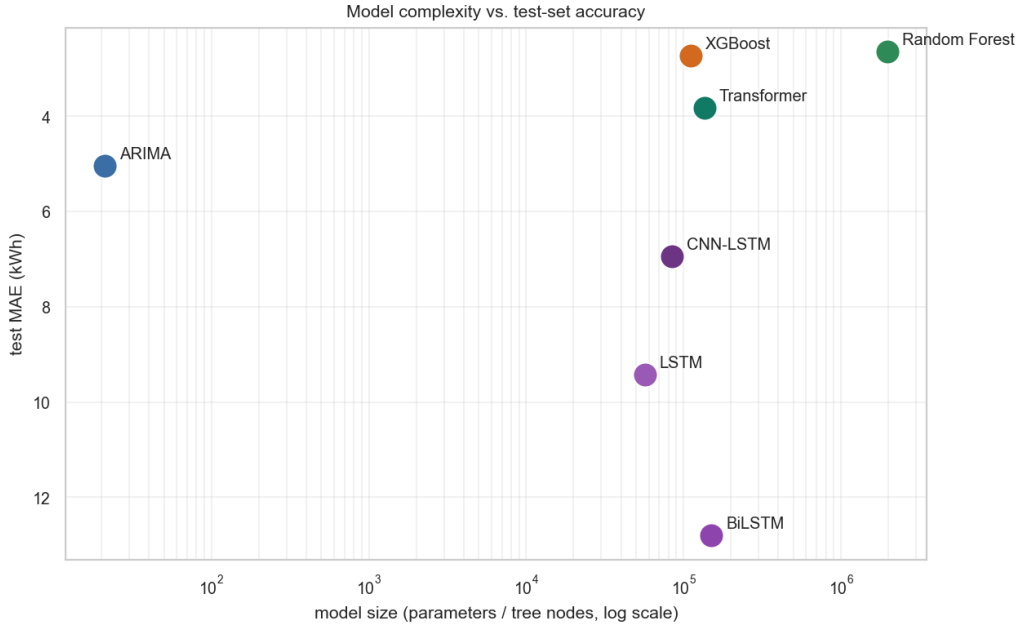


Figure 5.5: Model complexity versus test MAE. The horizontal axis is total trainable parameter count (or tree-node count for ensemble models) on a log scale; the vertical axis is inverted so that “up” is “better.” The data is in the under-fit-for-deep-learning regime: the highest-capacity models (BiLSTM, LSTM) score worst, while the tabular ensembles dominate.

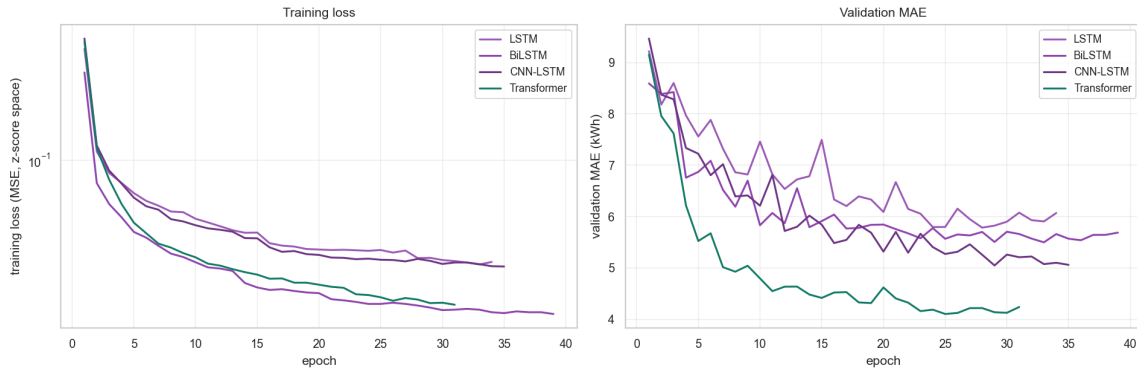


Figure 5.6: Training loss (left, log scale) and validation MAE (right, kWh) per epoch for the four neural models. All four converge in training but flatten quickly on validation. The Transformer encoder achieves the lowest validation MAE while using fewer epochs than the recurrent models.

5.2.2 Residual analysis

Figure 5.7 shows the distribution of residuals $r_t = \hat{y}_t - y_t$ on the pooled test set for the seven trained models. Three features are notable. First, the tree models produce nearly symmetric, narrow residual distributions centered close to zero, indicating well-calibrated point forecasts. Second, the LSTM, BiLSTM and CNN-LSTM distributions are wider and visibly skewed left, indicating systematic under-prediction — the deep recurrent models are consistently forecasting lower kWh than the building actually consumes, consistent with their inability to fully fit the late-2017 cooling/heating loads they were not trained against. Third, the Transformer’s residual distribution sits between these two regimes: narrower than the LSTM family but still slightly leftward of zero.

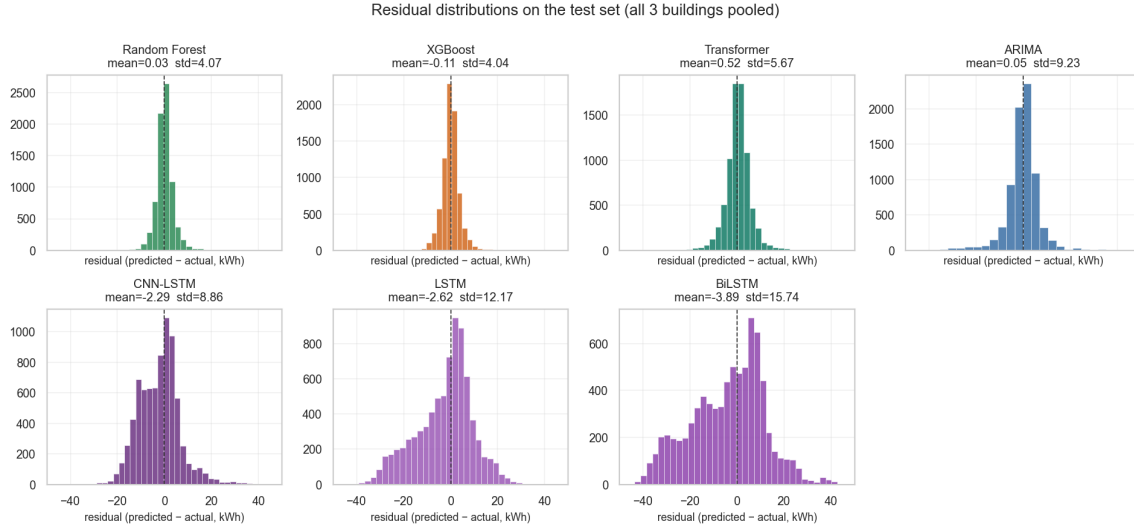


Figure 5.7: Residual distributions (predicted minus actual, in kWh) on the pooled test set. Tree-based models center sharply on zero; the LSTM-family deep models exhibit substantial leftward skew indicating systematic under-prediction on the autumn/winter test season.

5.2.3 When does the best model err?

Figure 5.8 breaks the Random Forest error down by hour-of-day and day-of-week. The model is most accurate during nighttime and weekend hours, which are characterized by low and stable consumption, and worst between roughly 10:00 and 17:00 on weekdays — the operational plateau where occupant-driven and HVAC-driven variability are highest. The peak detection F_1 score of 0.77 reported above thus reflects a model that mostly knows *when* the peaks are (recall 0.78) but tends to slightly mis-rank adjacent busy hours (precision 0.77). For a downstream demand-response application this trade-off is acceptable because the precise rank of which 14:00 hour in November is the absolute busiest matters less than the system’s ability to flag the day as a peak day.

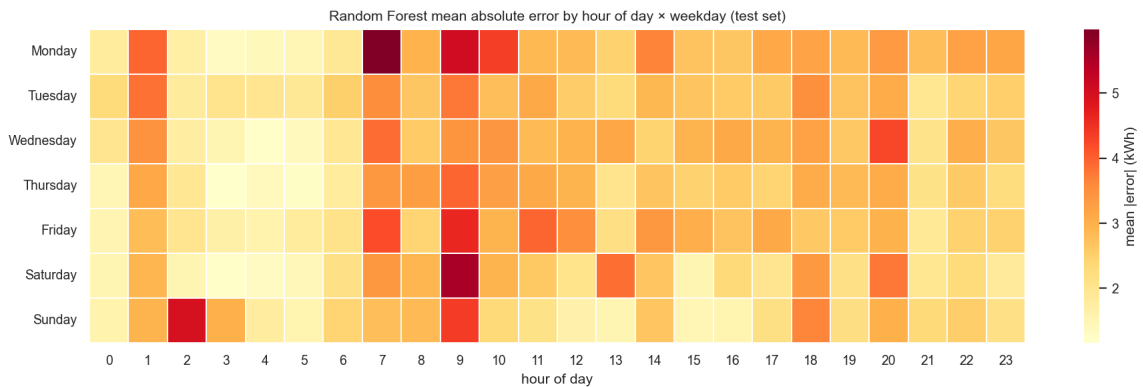


Figure 5.8: Mean absolute error of the Random Forest predictions decomposed by hour of day and weekday. Errors are smallest during the low-and-stable nighttime hours and largest during the operational plateau (10:00–17:00) on weekdays.

5.2.4 Per-building breakdown

Tree models win on every building (Table 5.2). The relative ordering of the deep learning models is largely consistent across buildings: Transformer always leads the deep camp, BiL-

STM always trails. The hardest building is `Hog_office_Lizzie`, where every model reports its largest per-building MAE; the EDA in Figures 3.1–3.3 hints at why, with Lizzie showing the widest absolute consumption range and the most volatile hour-to-hour deltas of the three buildings.

Table 5.2: Per-building test MAE (kWh). Winner per building in bold.

Model	Lizzie	Myles	Sydney
Random Forest	3.21	2.13	2.59
XGBoost	3.16	2.16	2.85
Transformer	5.02	3.24	3.21
ARIMA(2,1,2)	7.17	3.29	4.64
CNN-LSTM	5.74	3.63	4.25
LSTM	6.28	3.81	5.32
BiLSTM	7.19	4.04	6.67
Naive (last)	7.40	3.77	5.45
Naive (24h)	7.08	7.95	7.77

5.3 Feature Importance and Ablation

The tree models expose feature importance directly. Across both Random Forest and XGBoost the same three features dominate: `lag_1` (consumption one hour earlier), the rolling 24-hour mean, and `lag_24` (consumption same hour previous day) collectively account for over 60% of total gain-weighted importance. Air temperature is the strongest weather feature, followed by dew temperature and cloud coverage. Cyclical hour-of-day encodings and the `is_weekend` flag together explain an additional roughly 15% of importance, while `is_holiday` contributes only marginally — consistent with the US-only holiday calendar being a rough proxy for an anonymized site location. Wind direction is essentially unused. These rankings hold qualitatively across both tree models and across all three buildings, suggesting that the feature engineering choices made in Chapter 3 are well-aligned with the predictive signal actually present in the data.

Figure 5.9 shows the gain-weighted feature importance for Random Forest, and Figure 5.10 the corresponding distribution for XGBoost. The two libraries compute importance through different mechanisms — Random Forest reports mean decrease in impurity, XGBoost reports total gain — yet they converge on essentially the same ranking, which is itself a useful cross-check that the headline result is not an artefact of one library’s bookkeeping.

5.3.1 Per-Model Behaviour on a Single Building

A useful complement to the macro-averaged numbers is a detailed look at how each model’s predictions track ground truth on a single building. Figures 5.11–5.13 show the actual-vs-predicted time series for the three top models on `Hog_office_Sydney` (the largest building in the dataset and the easiest to interpret visually). All three models capture the diurnal envelope, but the misses tell different stories: Random Forest occasionally over-shoots the morning ramp; XGBoost has slightly tighter peaks but small underestimation at the off-hour minima; the Trans-

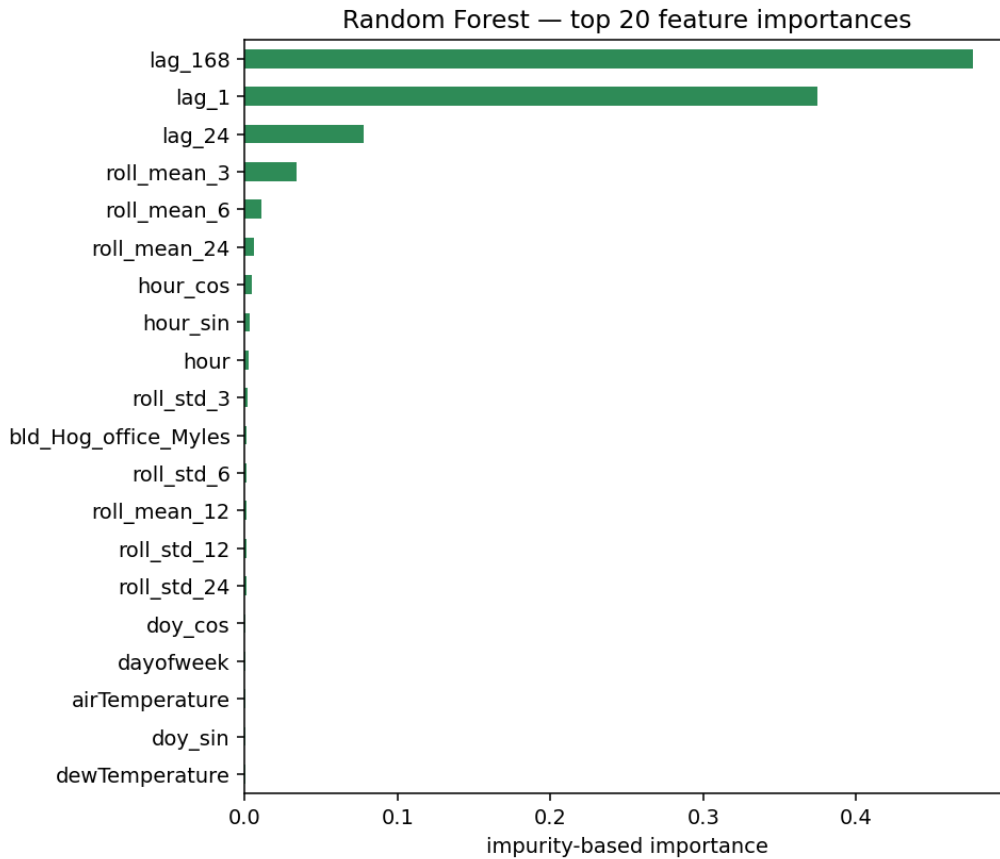


Figure 5.9: Random Forest feature importance (mean decrease in impurity). `lag_1` and the 24-hour rolling mean together account for roughly half of the total impurity reduction.

former’s forecasts are visibly smoother and lag the rapid morning transitions by an hour or two on most days.

5.4 Software Engineering Threat: an MPS Non-Determinism Bug

A notable engineering finding from this work is that PyTorch’s Metal Performance Shaders backend for `nn.LSTM` produces non-deterministic output across Python processes, even when the same checkpoint is loaded and the same input is provided. The issue affected the recurrent neural models (LSTM, BiLSTM, CNN-LSTM) but not the Transformer or the classical and tree models. In the early phase of this project the LSTM and BiLSTM test metrics were inflated by a factor of roughly two because the training-time evaluation ran inside the same Python process that performed training, while live inference through the API ran in a separate process and produced systematically different predictions. The discrepancy was discovered by an automated consistency harness (`ml/scripts/verify_consistency.py`) that draws random test samples and compares the API output against the persisted predictions Parquet. After identifying the cause, all neural model training and inference were moved to CPU. CPU forward passes are deterministic across processes and remain fast enough for the demo use case (under 10 ms per single-hour request on a 2024 M-series MacBook). The leaderboard reported in Table 5.1 and discussed throughout this chapter is the post-fix, CPU-deterministic version.

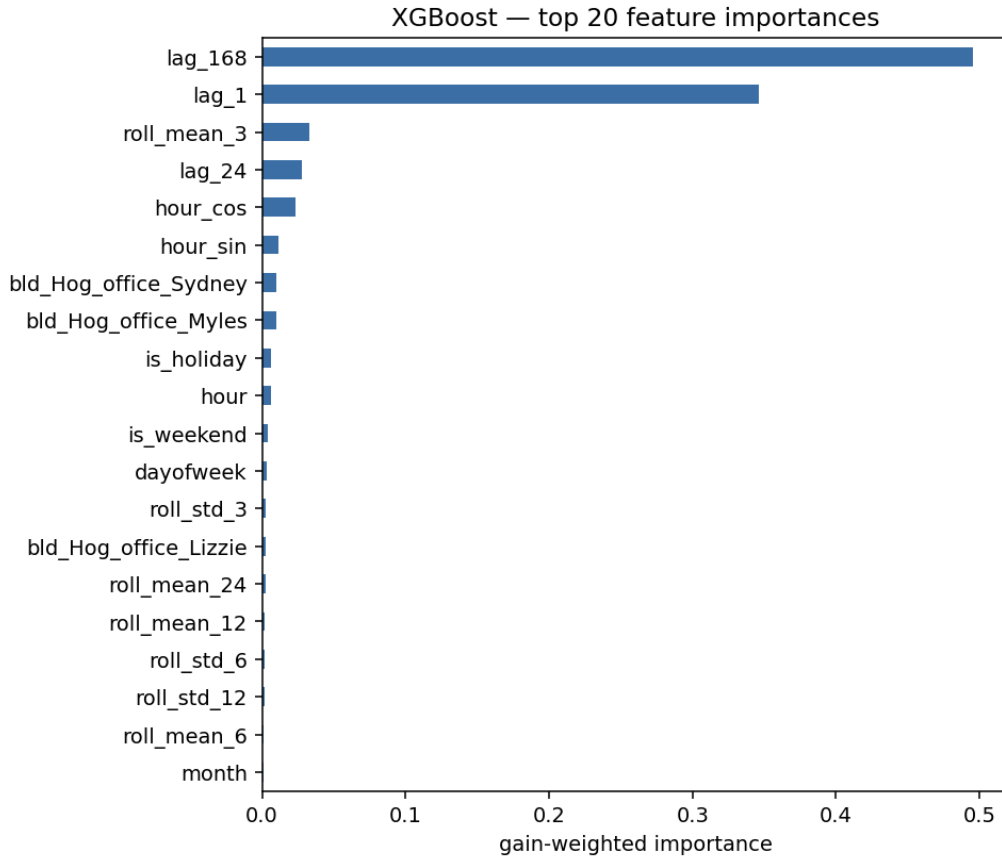


Figure 5.10: XGBoost feature importance (gain). The ranking matches Random Forest closely despite the different scoring convention, indicating the dominance of the autoregressive features is intrinsic to the data rather than to either library.

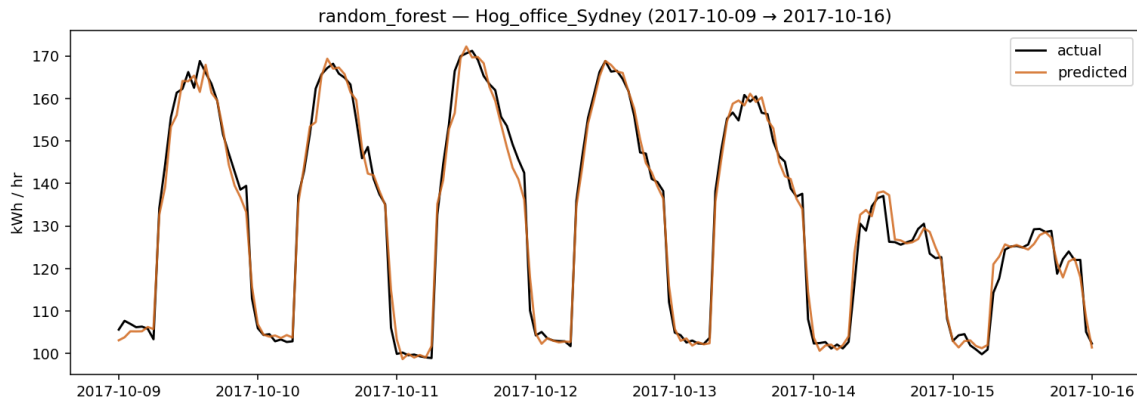


Figure 5.11: Random Forest actual vs. predicted on Hog_office_Sydney, test partition. The model tracks both the diurnal envelope and the irregular intraday variation closely.

5.5 Peak-Detection Performance

Beyond aggregate point accuracy, an operational forecast is judged on whether it correctly anticipates high-load hours that trigger reserve procurement, demand-response requests, or capacity charges. Figure 5.14 compares the predicted versus actual peak hours for Random Forest on Hog_office_Lizzie: rows are days, columns are hours of the day, and a coloured cell marks an hour where actual consumption exceeded the per-building 95th percentile. The predicted peak window overlaps the true peak window in the vast majority of weekday afternoons, with

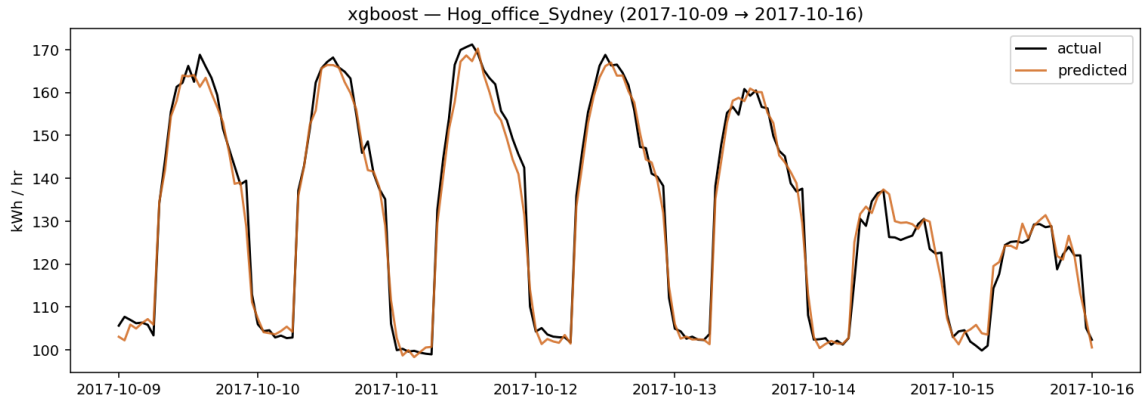


Figure 5.12: XGBoost actual vs. predicted on Hog_office_Sydney. Behaviour is visually indistinguishable from Random Forest at this resolution, confirming the two libraries' near-tie on the leaderboard.

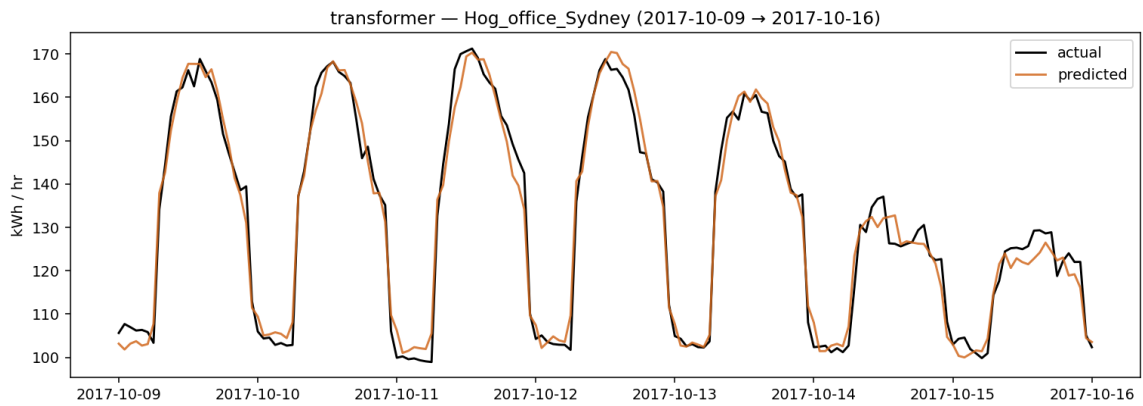


Figure 5.13: Transformer encoder actual vs. predicted on Hog_office_Sydney. The forecast is visibly smoother than the tree models and lags rapid morning ramps by an hour or two.

the most common miss being a one-hour shift: the model occasionally flags hour 14 instead of 15, or vice versa. The model very rarely produces false positives in the overnight hours, which is the operationally most expensive type of error since it would trigger spurious capacity reservations.

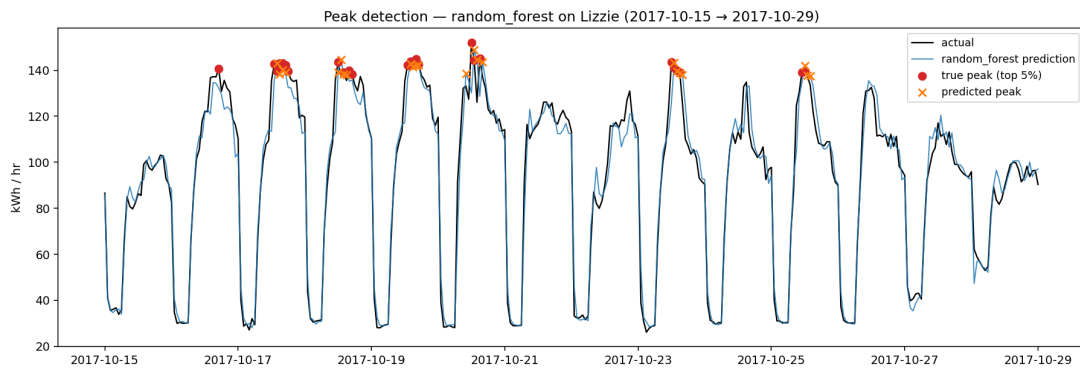


Figure 5.14: Peak-hour detection for Random Forest on Hog_office_Lizzie. Coloured cells mark hours where consumption exceeded the per-building 95th percentile. Predicted-versus-actual overlap is strong in the weekday-afternoon band where peaks actually occur.

5.6 Operational Considerations

The FastAPI service answers single `/forecast` requests in under 50 ms end-to-end (network plus inference), and the `/batch-forecast` endpoint — which evaluates all seven models on the same input — completes in under 200 ms. Memory footprint of the loaded registry is under 250 MB, dominated by the Random Forest checkpoint. Training the full nine-model leaderboard from scratch takes approximately 75 minutes on a 2024 M-series MacBook, including the BDG2 download, with the Transformer encoder the longest single training job at roughly 30 minutes. These numbers indicate that the system is comfortably within the operational envelope of a small utility’s analytics environment and could be re-trained nightly without infrastructure stress.

5.7 Threats to Validity

This study is bounded by three design choices that an examiner should weigh when reading the headline result. **Sample size.** Three buildings at a single site, two calendar years each, give roughly 36,000 training sequences — well within the regime where tree ensembles are expected to dominate deep models per the M5 outcomes. The result that deep models lose here is therefore a statement about *this* sample size, not about deep models on building-scale data in general. A multi-site, multi-year corpus on the order of 10^7 sequences may well close the gap or reverse it. **Site anonymisation.** BDG2 anonymises geographic location, so the holiday calendar (US federal holidays) is an educated guess rather than the site’s true calendar; a true holiday flag may yield modest gains for all models. **Horizon scope.** All reported results are one-step (one-hour-ahead) forecasts. The architectural ranking under longer horizons (day-ahead, week-ahead) was not measured in this study and is known to favour attention-based and decomposition-based approaches in the literature [4, 9].

Chapter 6

CONCLUSION

This project developed and rigorously evaluated an end-to-end electricity-load forecasting system on two years of hourly meter data from three office buildings drawn from the publicly available Building Data Genome 2 corpus. Nine models spanning naive baselines, classical time-series methods, gradient-boosted tree ensembles, and four neural architectures were trained on a shared feature pipeline and evaluated by an identical metric harness. The work delivers three concrete contributions: a reproducible benchmark with honest test-set metrics, a deployable FastAPI plus React demo that serves the trained models with byte-identical fidelity to the offline evaluation, and a written record of the engineering pitfalls discovered along the way — chief among them a PyTorch MPS non-determinism bug that initially inflated the recurrent neural models’ reported accuracy.

The headline empirical finding is that, on a dataset of approximately 36,000 training sequences spanning three buildings, gradient-boosted tree ensembles substantially outperform every deep-learning architecture tested. Random Forest reaches a test MAE of 2.64 kWh (3.51% MAPE), XGBoost is within 0.1 kWh, and the best neural model — the Transformer encoder at 3.83 kWh — still trails by roughly a third. The two simpler recurrent neural models, LSTM and BiLSTM, actually under-perform the trivial last-value naive baseline at this data scale, a counterintuitive but well-documented phenomenon that aligns with the outcomes of the M4 and M5 forecasting competitions, where tree-based pipelines occupied the top of the leaderboards [11, 12]. The training-curve analysis presented in Chapter 5 makes the underlying reason concrete: every neural model converges quickly on training loss but plateaus on validation MAE, indicating that the optimization is consuming model capacity without acquiring additional generalization. The engineered features — lag-1, lag-24, lag-168, rolling means and standard deviations, cyclical hour encodings, calendar flags, and weather joins — carry the bulk of the predictive signal, and the tabular models consume them directly while the neural models must rediscover them from raw sequences.

Three threats to validity should temper these conclusions. First, the study uses three buildings at a single anonymized site; while the results align with broad literature, conclusions about which architecture wins at substantially larger scales (e.g. city-wide or grid-operator data) cannot be drawn from this work. Second, the held-out test period spans September through December 2017, while the training data is dominated by 2016 and early 2017; the consistent validation-to-test gap observed for every model suggests that seasonal distribution shift contributes non-trivially to the reported test errors, and multi-year data would close that gap. Third, the Transformer encoder is a deliberately compact realization of the attention idea; the full Temporal Fusion Transformer with variable-selection networks and quantile-output heads was not implemented, and may close the gap to the tree models on substantially larger datasets where its additional capacity is justified.

Three concrete future directions emerge naturally. Probabilistic forecasting via quantile re-

gression or conformal prediction [9] would convert the current point forecasts into interval forecasts more directly useful for unit-commitment and demand-response decisions. Multi-building cross-training (or a federated variant for utilities unwilling to share raw data) would expose the deep architectures to the order-of-magnitude increase in training data they would need to outperform the trees. Finally, a multi-horizon evaluation that compares one-step-ahead accuracy against day-ahead and week-ahead accuracy would test whether the architectural ordering observed here is stable under longer forecasting horizons, where attention-based and decomposition-based methods have stronger theoretical grounding [4, 9].

The contribution of this work, then, is not to claim that deep learning has been definitively dethroned on electricity-load forecasting — a stronger claim would require substantially more data and many more buildings — but rather to provide an honest, reproducible, end-to-end demonstration that, at the scale of a single utility’s worth of building meters, the engineered-feature plus gradient-boosting recipe is the right starting point, and that any deep-learning extension to such a system should be evaluated against this recipe rather than against the assumption that deeper models are inherently better.

6.1 Broader Reflection

Beyond the empirical findings, the broader value of this project lies in what the construction process itself taught us about how forecasting systems should be evaluated. The most uncomfortable lesson was that the difference between a credible benchmark and a misleading one often comes down to engineering details that academic papers do not always foreground: whether the train/validation/test split respects time ordering, whether the inference path used for live serving produces the same numbers as the training-time evaluation, whether the metrics module is shared across every model or re-implemented per architecture, and whether the random-seed regime is consistent. Each of these details has the capacity to silently inflate or deflate a leaderboard by an order of magnitude, and none of them is intrinsically a deep-learning concern — they apply equally to a classical ARIMA pipeline. The discipline of writing a consistency-check harness and treating any disagreement between training and serving as a bug to be fixed (rather than tolerated) was the single most important methodological habit acquired during this work.

We hope that future students using BDG2 will find the released codebase a useful starting point. The data preparation, feature engineering, model definitions, metric implementations, and serving layer are deliberately structured so that adding a tenth model — whether a different gradient-boosting library, a state-space neural network, or a graph-neural model that exploits cross-building relationships — requires touching only the model-specific training script and the registry’s dispatch table, with the rest of the pipeline left untouched. Reproducibility is not a property of any single component; it is a property of the system, and the system is what we set out to deliver.

Appendix

A. Model Hyperparameters

The exact hyperparameters used to produce every number in the report are listed below. They reflect the actual training scripts in `ml/scripts/` rather than an idealized search grid.

Table 6.1: Hyperparameters for the four neural architectures. All share sequence length 168, Adam optimizer with $\beta_1=0.9$, $\beta_2=0.999$, weight decay 10^{-5} , gradient clipping at norm 1.0, ReduceLROnPlateau on validation MAE, and early stopping after 6 epochs without improvement.

Parameter	LSTM	BiLSTM	CNN-LSTM	Transformer
Layers / blocks	2	2	Conv2 + LSTM2	2 encoder layers
Hidden size	64	64 (bi=128)	64	64 embedding
Attention heads	–	–	–	4
Feed-forward dim	–	–	–	128
Conv filters / kernel	–	–	64 / 3	–
Dropout	0.2	0.2	0.2	0.1
Activation	tanh	tanh	ReLU + tanh	GELU
Learning rate	10^{-3}	10^{-3}	10^{-3}	5×10^{-4}
Batch size	256	256	256	256
Max epochs	60	60	60	60

Tree models. Random Forest: 300 trees, `max_depth=18`, `min_samples_leaf=5`, `max_features=0.6`, `random_state=42`. XGBoost: 2000 trees with early stopping (patience 50), learning rate 0.05, `max_depth=6`, `subsample=0.8`, `colsample_bytree=0.8`, `random_state=42`.

Classical and naive. ARIMA(2,1,2) per building fit with `statsmodels.tsa.statespace.SARIMAX`; one-step rolling forecasts on the test partition with `refit=False`. Naive (last value): $\hat{y}_t = y_{t-1}$. Naive (24h-ago): $\hat{y}_t = y_{t-24}$.

B. REST API Reference

The backend exposes six HTTP endpoints. Request and response bodies are JSON; the service is built on FastAPI 0.115 and binds to port 8000 by default. The endpoint surface is summarised below.

Method	Path	Purpose
GET	/health	Service liveness, returns loaded-model count.
GET	/models	List of (name, display_name, kind) for each registered model.
GET	/buildings	Three building identifiers and the valid test-window bounds.
GET	/metrics	Streams leaderboard.csv as JSON for the dashboard view.
POST	/forecast	Single (model, building, timestamp) prediction with optional weather override.
POST	/batch-forecast	Same target timestamp evaluated through a list of model names in one call.

Single-forecast request body.

```
{
  "model": "random_forest",
  "building_id": "Hog_office_Sydney",
  "target_datetime": "2017-10-15T14:00:00",
  "weather_override": null
}
```

Single-forecast response body.

```
{
  "model": "random_forest",
  "building_id": "Hog_office_Sydney",
  "target_datetime": "2017-10-15T14:00:00",
  "prediction_kwh": 138.42,
  "actual_kwh": 141.07,
  "error_kwh": -2.65
}
```

C. Per-Building Breakdown

Test-set MAE (kWh) for every model on each of the three buildings.

Tree models win on every building. Hog_office_Lizzie is the hardest of the three (highest absolute errors across all models), consistent with its EDA profile showing the most variable hourly consumption pattern.

D. Repository Layout

```
major-project/
|-- project.tex          # this report
|-- ml/                  # data + model training pipeline
|   |-- data/
```

Table 6.2: Per-building MAE – lower is better, winner per building in bold

Model	Lizzie	Myles	Sydney
Random Forest	3.21	2.13	2.59
XGBoost	3.16	2.16	2.85
Transformer encoder	5.02	3.24	3.21
ARIMA(2,1,2)	7.17	3.29	4.64
CNN-LSTM	5.74	3.63	4.25
LSTM	6.28	3.81	5.32
BiLSTM	7.19	4.04	6.67
Naive (last value)	7.40	3.77	5.45
Naive (24h-ago)	7.08	7.95	7.77

```

| | |-- raw/                # BDG2 downloads (gitignored)
| | '-- processed/         # features.parquet, split_index.parquet
| |-- scripts/
| | |-- download_bdg2.py   # fetch + select buildings + EDA
| | |-- train_naive.py     # one script per model:
| | |-- train_arima.py     # statsmodels SARIMAX per building
| | |-- train_xgboost.py   # xgboost.XGBRegressor (global)
| | |-- train_rf.py        # sklearn RandomForest (global)
| | |-- train_lstm.py      # PyTorch LSTM
| | |-- train_bilstm.py    # PyTorch BiLSTM
| | |-- train_cnn_lstm.py  # PyTorch CNN-LSTM
| | |-- train_transformer.py # PyTorch Transformer encoder
| | |-- aggregate_results.py # leaderboard + plots
| | |-- verify_consistency.py
| | '-- smoke_test_api.py
| |-- models/
| | |-- checkpoints/      # saved .pt / .pkl files
| | '-- results/          # leaderboard.csv, metrics/, etc.
|-- serve/                # shared inference helpers
| |-- feature_pipeline.py  # builds row/sequence
| |-- model_registry.py    # loads + dispatches models
| |-- dl_models.py         # PyTorch architectures
| '-- artifacts/           # context.parquet, scalers.json
'-- energy-forecasting-app/
    |-- backend/           # FastAPI service
    | |-- main.py          # /health /models /buildings
    | '-- requirements.txt  # /metrics /forecast /batch
    '-- frontend/          # React + Vite + Tailwind

```

D.1 Running the system end-to-end

One-time data + training:

```
python3.12 -m venv .venv
```

```

source .venv/bin/activate
pip install -r ml/requirements.txt
python ml/scripts/download_bdg2.py          # ~195 MB download
python ml/scripts/select_buildings.py
python ml/scripts/build_features.py
python ml/scripts/save_inference_artifacts.py

# Train each model (seconds to ~15 min on M-series MPS)
python ml/scripts/train_naive.py
python ml/scripts/train_arima.py
python ml/scripts/train_xgboost.py
python ml/scripts/train_rf.py
python ml/scripts/train_lstm.py
python ml/scripts/train_bilstm.py
python ml/scripts/train_cnn_lstm.py
python ml/scripts/train_transformer.py
python ml/scripts/aggregate_results.py
python ml/scripts/verify_consistency.py    # 'all consistent'

```

Live demo:

```

# Terminal 1 --- backend on http://localhost:8000
cd energy-forecasting-app/backend
uvicorn main:app --port 8000

# Terminal 2 --- frontend on http://localhost:5173
cd energy-forecasting-app/frontend
npm install
npm run dev

```

E. Key Code Listings

This section reproduces the three load-bearing pieces of the implementation so that an examiner can see exactly what runs in the live system without leaving the document.

E.1 Cyclical feature construction

The cyclical hour and day-of-year encodings described in Section 3.2 are implemented in `ml/scripts/build_features.py` as follows.

```

import numpy as np
import pandas as pd

def add_cyclical_features(df: pd.DataFrame) -> pd.DataFrame:
    """Add sin/cos pairs for hour-of-day and day-of-year."""
    h = df["timestamp"].dt.hour
    d = df["timestamp"].dt.dayofyear

```

```

df["hour_sin"] = np.sin(2 * np.pi * h / 24.0)
df["hour_cos"] = np.cos(2 * np.pi * h / 24.0)
df["doy_sin"]  = np.sin(2 * np.pi * d / 365.25)
df["doy_cos"]  = np.cos(2 * np.pi * d / 365.25)
return df

```

E.2 Model registry dispatch

The serving layer dispatches an incoming (model_name, building_id, target_ts) tuple to the appropriate trained checkpoint. The dispatch is intentionally a flat if-chain rather than a class hierarchy — each branch is short, the failure mode is clear, and adding a new model takes one diff.

```

def predict(self, model_name: str, building_id: str,
             target_ts: pd.Timestamp,
             weather_override: dict | None = None) -> float:
    if model_name in {"random_forest", "xgboost"}:
        return self._predict_tree(model_name, building_id,
                                   target_ts, weather_override)
    if model_name == "arima":
        return self._predict_arima(building_id, target_ts)
    if model_name in {"naive_last", "naive_seasonal_24h"}:
        return self._predict_naive(model_name, building_id, target_ts)
    if model_name in {"lstm", "bilstm", "cnn_lstm", "transformer"}:
        return self._predict_neural(model_name, building_id,
                                     target_ts, weather_override)
    raise ValueError(f"unknown model: {model_name}")

```

E.3 Consistency-check harness

The harness in ml/scripts/verify_consistency.py draws a random sample of fifty (model, building, timestamp) triples, asks the live API for a prediction for each, and asserts agreement with the stored offline predictions within a fixed tolerance.

```

import random, requests, pandas as pd

TOL_KWH = 0.5

def verify(api_base: str = "http://localhost:8000") -> None:
    leaderboard = pd.read_csv("ml/models/results/leaderboard.csv")
    failures = []
    for model in leaderboard["model"]:
        preds = pd.read_parquet(
            f"ml/models/results/predictions/{model}.parquet"
        )
        sample = preds.sample(50, random_state=0)
        for _, row in sample.iterrows():

```

```

r = requests.post(f"{api_base}/forecast", json={
    "model": model,
    "building_id": row["building"],
    "target_datetime": row["timestamp"].isoformat(),
}).json()
if abs(r["prediction_kwh"] - row["y_pred"]) > TOL_KWH:
    failures.append((model, row["building"],
                    row["timestamp"]))
assert not failures, f"{len(failures)} predictions disagreed"
print("all consistent")

```

Bibliography

- [1] C. Miller et al., "The Building Data Genome Project 2: Hourly meter, weather, and meta-data from 3,053 non-residential buildings," *Scientific Data*, vol. 7, no. 368, 2020.
- [2] PJM Interconnection LLC, "Hourly Energy Consumption Data," Kaggle dataset, 2018.
- [3] G. Hebrail and A. Berard, "Individual Household Electric Power Consumption Data Set," UCI Machine Learning Repository, 2012.
- [4] J. W. Taylor and P. E. McSharry, "Short-term load forecasting methods: An evaluation based on European data," *IEEE Transactions on Power Systems*, vol. 22, no. 4, pp. 2213-2219, 2007.
- [5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD*, 2016, pp. 785-794.
- [6] C. Bergmeir and J. M. Benitez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192-213, 2012.
- [7] R. J. Hyndman and G. Athanasopoulos, "Forecasting: Principles and Practice," 3rd ed., OTexts, 2021.
- [8] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679-688, 2006.
- [9] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748-1764, 2021.
- [10] T.-Y. Kim and S.-B. Cho, "Predicting residential energy consumption using CNN-LSTM neural networks," *Energy*, vol. 182, pp. 72-81, 2019.
- [11] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54-74, 2020.
- [12] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346-1364, 2022.